



Tencent
腾讯学堂

《英雄联盟手游》 服务器基础架构的迭代和优化

jerrykong

LEARNING EMPOWERS YOUR FUTURE

分享人介绍



- Jerrykong, 中文名: 孔荣华
- 逍遥游工作室后台技术专家。
- 2008年加入腾讯。
- 经历过的重大项目: 《九界》、《全民飞机大战》、《英雄联盟手游》。



2008~2012

- 《九界》MMORPG项目
- 应用组组长, 负责多个业务系统的开发, 基础库FCL的优化维护, 性能优化等工作。

2014~2015

- 《全民飞机大战》休闲手游项目
- 服务器核心开发, 负责PK系统, 双打系统, 反外挂, 公会等多个核心业务系统开发。



2016~2023

- 《英雄联盟手游》MOBA手游项目
- 服务器主程, 负责团队管理, 基础架构搭建。

1

项目的背景

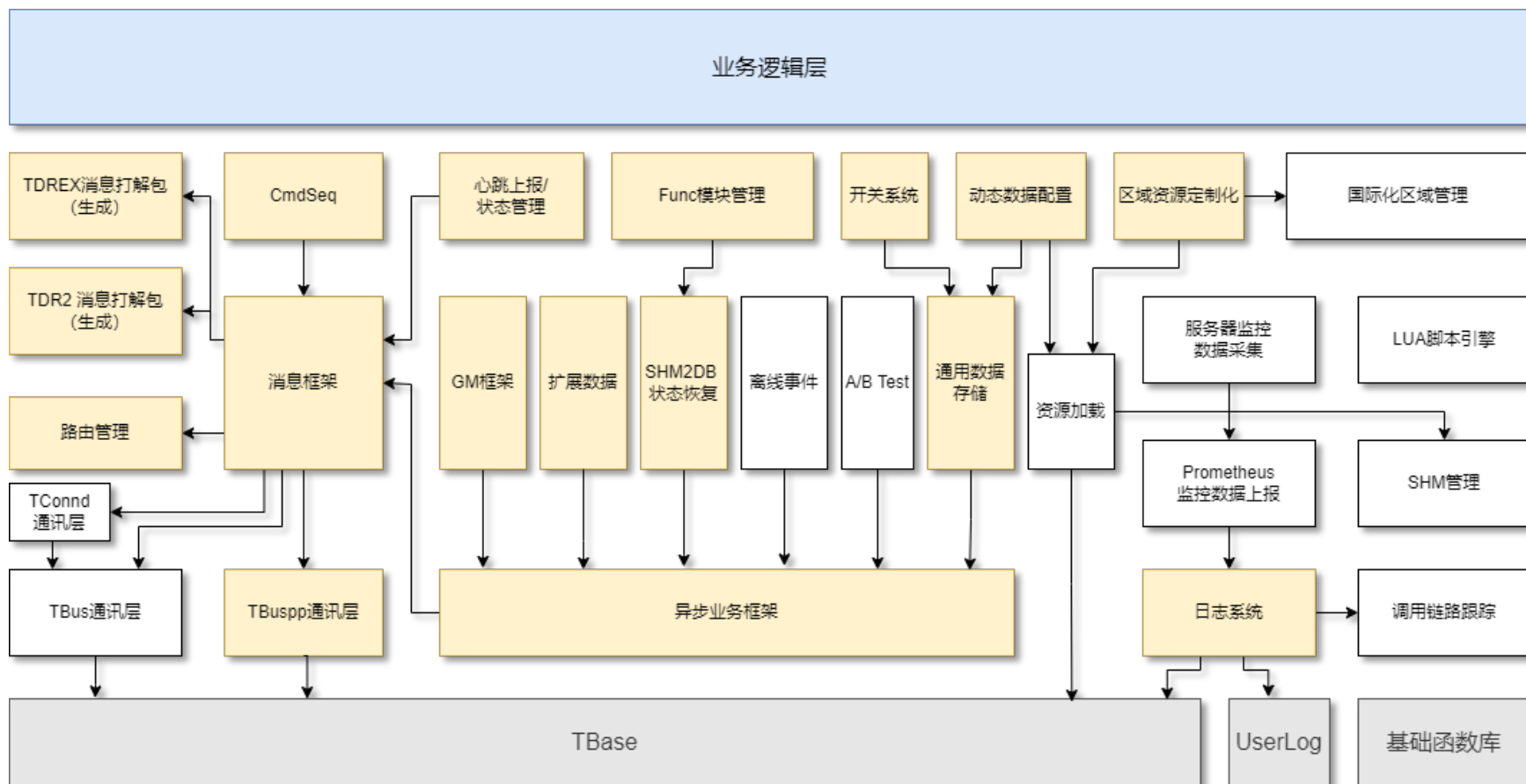
2

架构的演变

3

核心特性设计

基础组件架构图



项目的背景



《英雄联盟手游》代号“LGame”的背景：

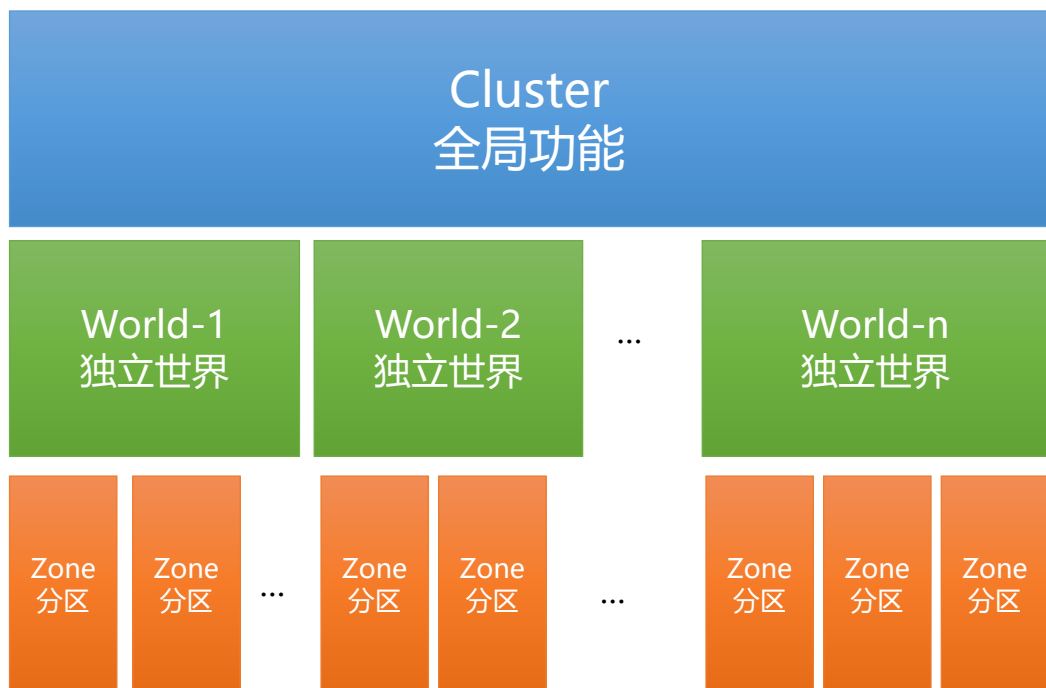
- MOBA品类玩法，多人在线PVP，单局玩法为主。
- 从 2016年4季度开始正式研发
- 以《全民飞机大战》的服务器核心代码为基础



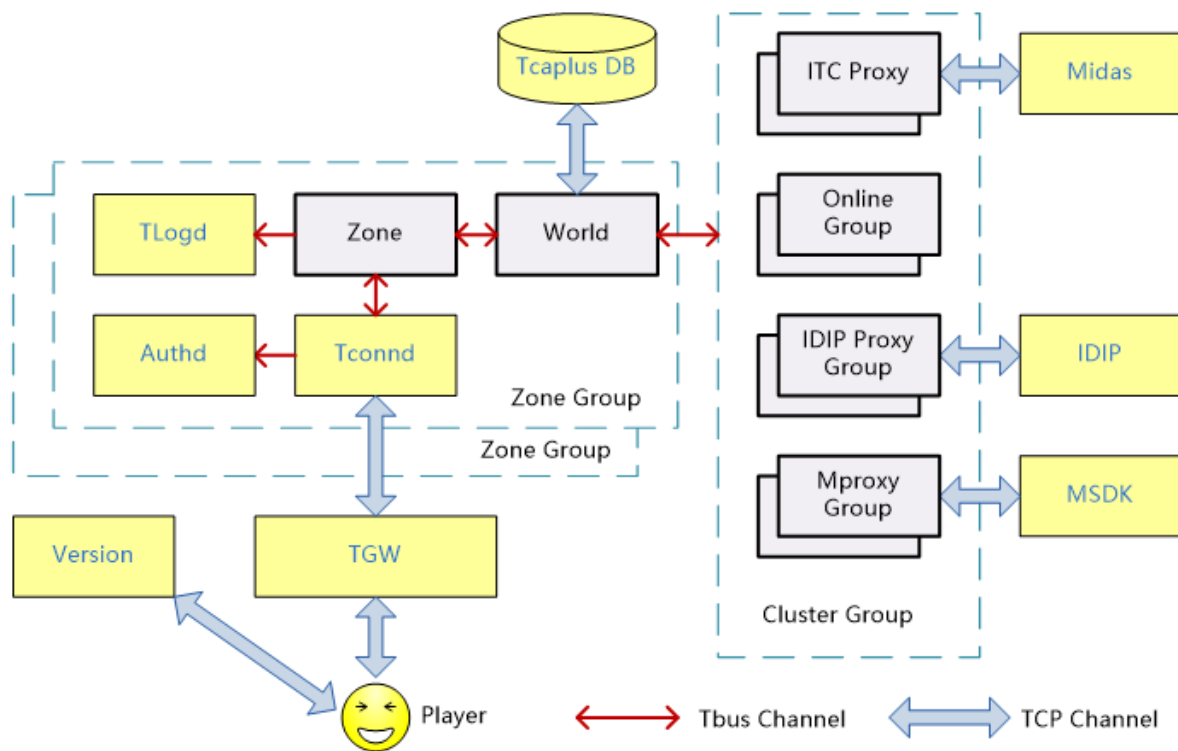
架构的演变 – 以飞机为基础

《飞机》的基础架构

- TSF4G框架
- 类3层架构 – 实际退化为二层
- TCM进程管理。



《飞机》的架构图:



- 业务逻辑集中，强耦合，冲突多。
- 业务的稳定性比较差。
- 扩展性不好，TCM难以实现灵活的运维方式。

架构的演变 – LGame的架构

架构的扁平化

底层通讯升级TBuspp

引入服务发现

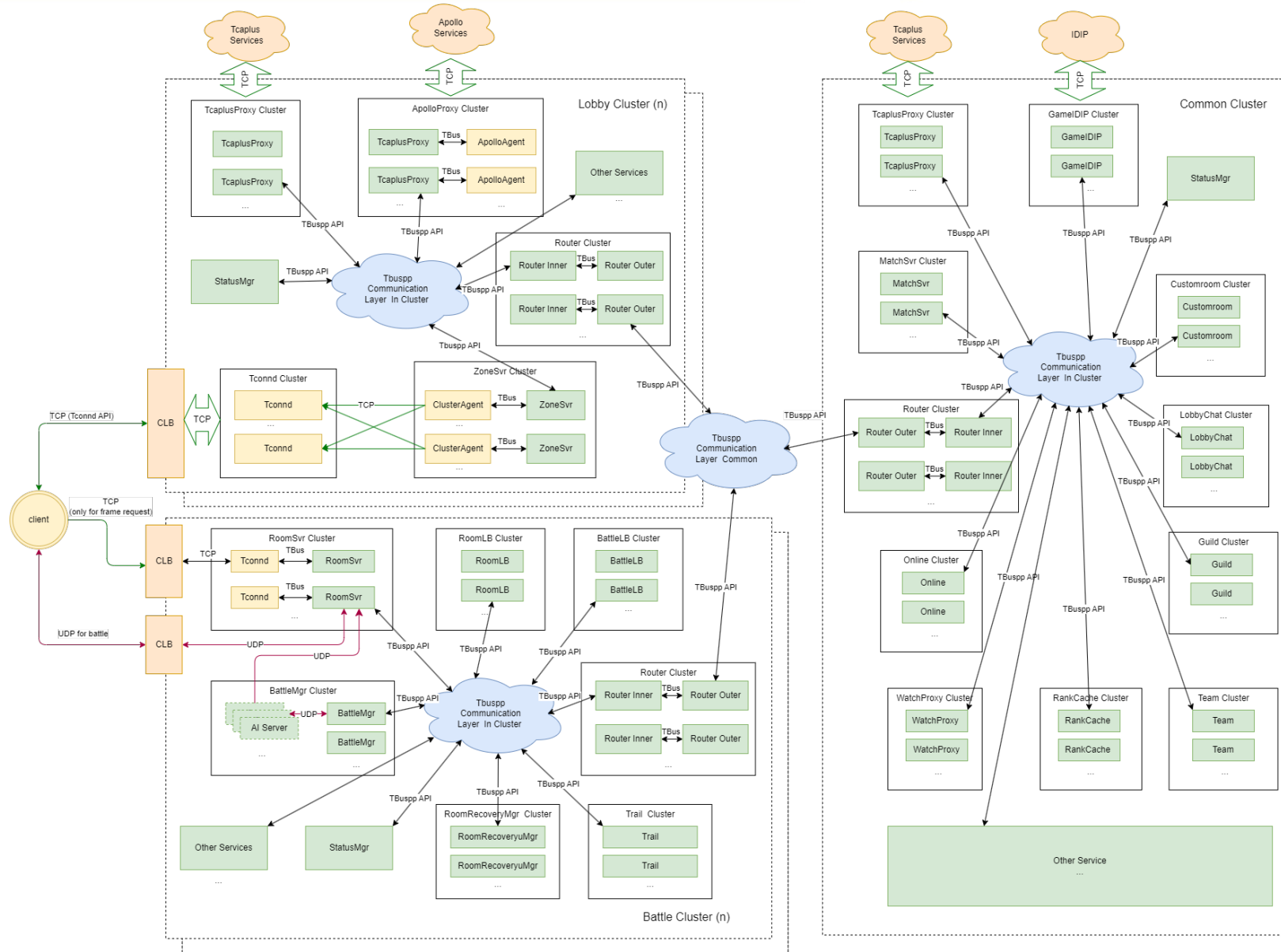
服务的容器化改造

消息和协议改造

引入服务状态管理

Tconnd集群化接入

➡ 向微服务化和上云的方向靠拢



架构的演变 – 从Tbus到Tbuspp

早期的Tbus集群解决方案:

- 结构简单，部署方便 – TCM自动配置。
- 基于SHM的透明跨进程通讯机制。
- 使用每个通道固定大小配置的SHM。

面临的问题

Server A

$n : m$

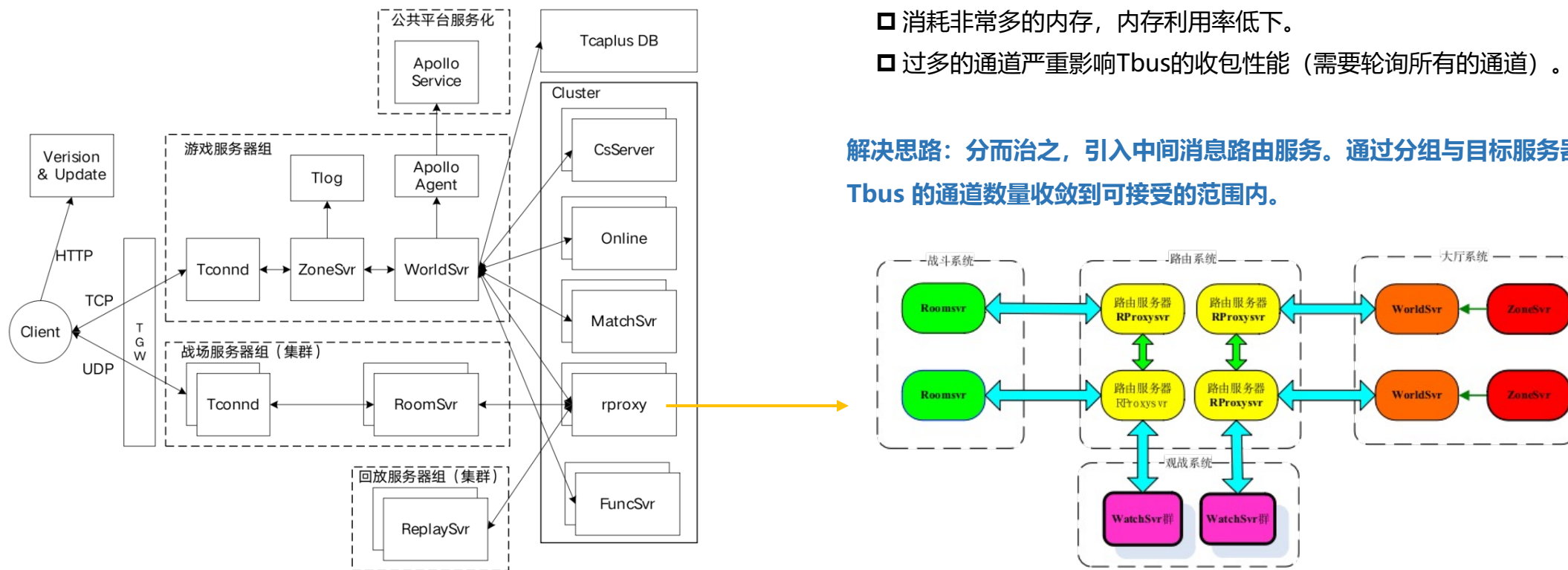
Server B

$Tbus \text{ 通道数} = n * m。$

如果n和m的数量都比较大的话，Tbus 通道的数量会非常多。面临的问题：

- ❑ 消耗非常多的内存，内存利用率低下。
- ❑ 过多的通道严重影响Tbus的收包性能（需要轮询所有的通道）。

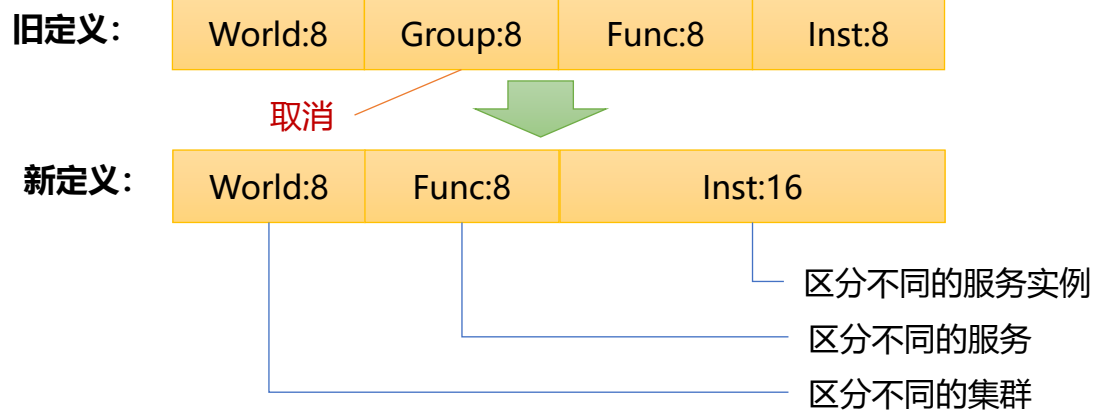
解决思路：分而治之，引入中间消息路由服务。通过分组与目标服务器连接，将Tbus 的通道数量收敛到可接受的范围内。



架构的演变 – 向Tbuspp的演进

控制改造成本，兼容BusID的路由方式，在此基础上拓展名字路由方式。

□ BusID的定义拓展

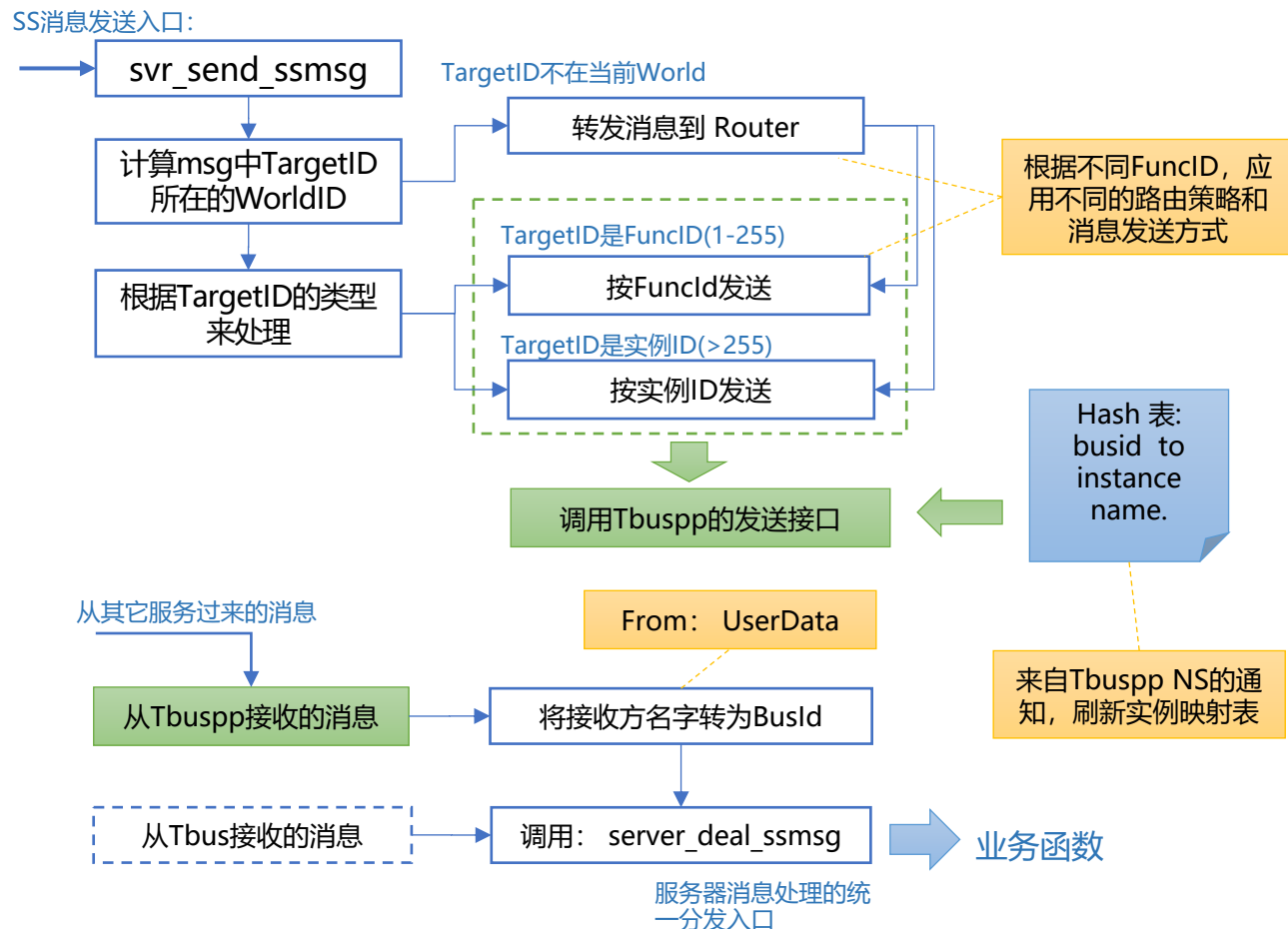


□ 封装Tbuspp的名字到BusID的映射



实例和BusID的关联：通过 UserData 来传递自身的BusID。通过Map来关联。

□ 封装消息的收发接口，消息路由等功能



架构的演变- 容器化改造

镜像构建和打包

- 升级以前的tgz打包和手工分发安装包的方式，修改为Docker镜像的方式。以适用容器化的部署。通过仓库来管理版本镜像，部署时直接从仓库拉取镜像到母机。

蓝鲸平台应用模板定义服务

- 通过蓝鲸平台的应用模板来配置各服务以及其资源需求,实例规模等。

ConfigMap下发应用配置

- 将不同环境不同服务的配置文件统一通过ConfigMap来管理，并在容器实例化时进行下发。替代原TCM的配置下发方式。

引入容器主进程管理器

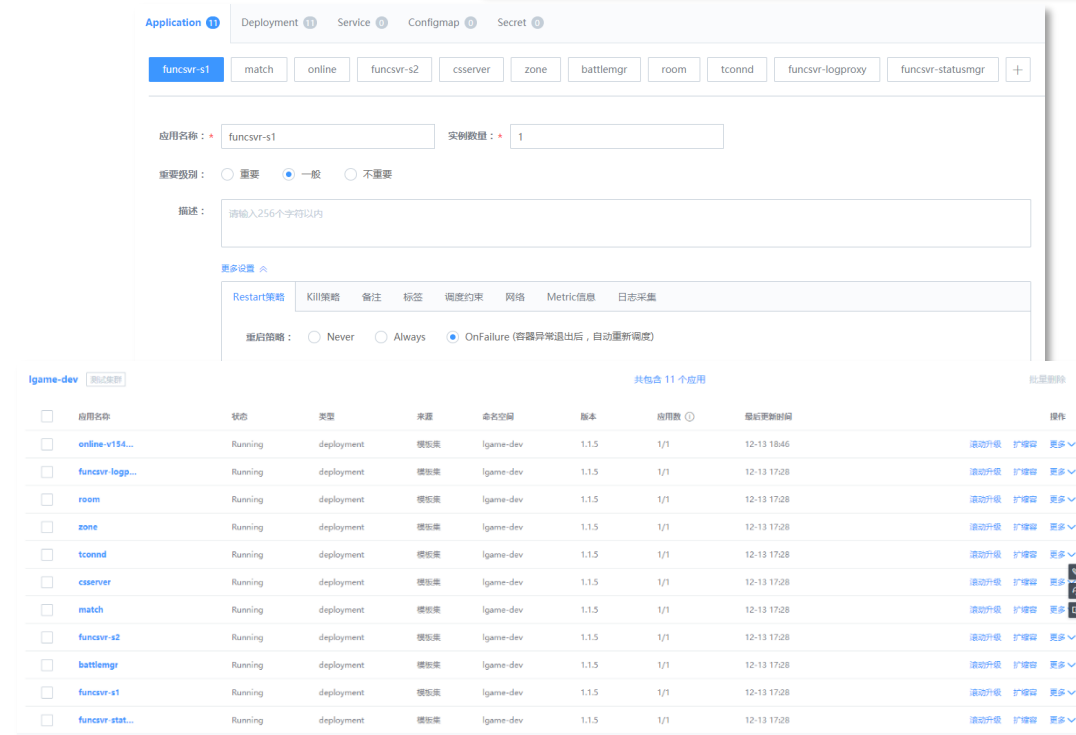
- 原服务启动后会存在多个进程需要相互配合,如主服务进程,日志进程,Tbuspp通讯进程等。为适应容器化的需要，对这些进程通过start.sh 进行统一管理和监控等，并将start.sh 作为容器的入口进程。同时支持容器内的局部进程或者配置热更，Crash后尝试自动拉起等。

配置生成从TCM->LCM: 自研的进程配置生成和管理工具

- 原TCMCFG只能在TCM进程管理器下运行，通过改造，将它移植到容器内，可以独立运行，根据下发的服务配置文件来动态生成各进程的运行时配置。

流水线的应用

- 为了方便运维操作和自动化。通蓝盾的流水线，将镜像的构建，服务的启停，升级，热更等。大大提升了运维的效率。



- 支持不同版本服务之间的兼容 – 不停服更新的基础
- 从TDR1 升级到TDR2成本更低，对业务改动最小
- 协议定义不用重写，业务不用修改

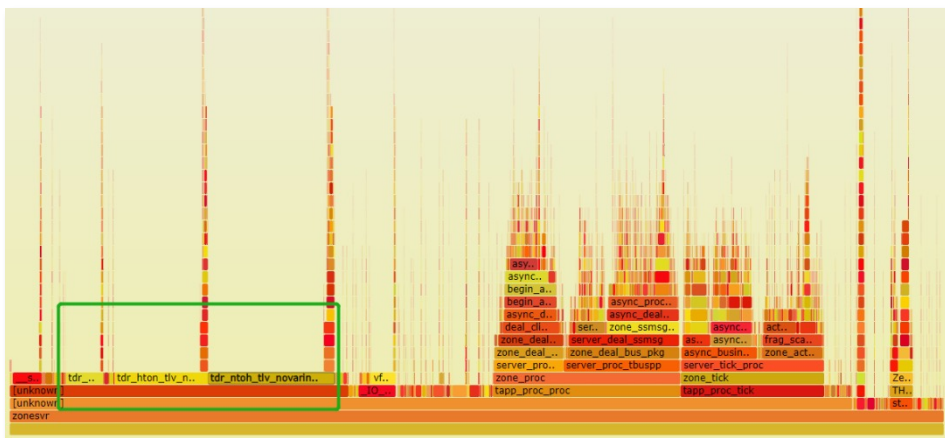
- TDR2 API 的打解包问题，代码存在BUG。
- 性能低 -> 自定义TDR2打解包代码生成。性能提升6~7倍。

而且按照这个代码来看，只要删除字段后面多加的那些字段都不会初始化；
按我们的修改思路应该是TDR_GET_NEXT_ENTRY_TLV_WITH_DEFAULT，但又单独写了一个TDR_GET_NEXT_ENTRY_TLV；
这个TDR_GET_NEXT_ENTRY_TLV就隐藏了比较多的问题

还要看下他们为什么要写这个TDR GET NEXT ENTRY TLV宏

[illegible]

这里打包有个Bug，打包 `wstring` 这种类型会出错。写错了，应该是用 `PACK_WSTRING_TYPE_ENTRY_TLV` 才对



```
int tdr2_pack_SSRoomRandomHeroS(const LPSSROOMRANDOMHEROS pstSSRoomRandomHeroS, char *pstData, int ilen, char chHiasMagicHead)
{
    int iRet = 0;
    char *pstOriginalDataPtr = pstData;
    if (chHiasMagicHead == 1) //写入MagicHead 0X99
    {
        if (pstData - pstOriginalDataPtr + sizeof(uint8_t) > ilen)
        {
            return tdxex_set_error(TDR2_AUTOCODE_ERR_OUT_OF_BUFFER, "SSRoomRandomHeroS", "", -1, "out of buffer while pack SSRoomRandomHeroS");
        }
        *(uint8_t*)pstData = 0X99;
        pstData += sizeof(uint8_t);
    }

    char *pstLenPtr = pstData; //长度头部位置
    pstData += sizeof(int32_t); //预留长度位置

    //编码字段 name:RoomID, type:uint64, id:1
    do
    {
        const static int iFieldId = 1;
        const static int iTypeId = TDR2_TYPE_ID_8_BYTE;
        uint32_t dwTag = (uint32_t)((iFieldId << 4) + iTypeId);
        int iUsed = 0;
        TDR_PACK_ID_TLV(uint32_t, dwTag, pstData, ilen - (pstData - pstOriginalDataPtr), iUsed, iRet);
        if (iRet < 0)
        {
            return tdxex_set_error(TDR2_AUTOCODE_ERR_OUT_OF_BUFFER, "SSRoomRandomHeroS", "RoomID", -1, "out of buffer while pack field name:RoomID");
        }
    }
}
```

优化后的性能对比

```

[roomsvr_40.12.1.log: [20221102 15:33:02.641809] [752] <DEBUG> [0] (room_on_db_query_result_update_round_result:65) 81294779803857126 [280c00043fc20012] [01000c280f0f6315] [01007e2844692235]
roomsvr_40.12.1.log: [20221102 15:33:02.721052] [753] <SYS> [0] (room_on_db_query_result_update_round_result:95) new unpack [79] ms [280c00043fc20012] [01000c280f0f6315] [01007e2844692235]
roomsvr_40.12.1.log: [20221102 15:33:03.256256] [754] <SYS> [0] (room_on_db_query_result_update_round_result:114) old unpack [536] ms [280c00043fc20012] [01000c280f0f6315] [01007e2844692235]
roomsvr_40.12.1.log_err: [20221102 15:33:02.721052] [753] <SYS> [0] (room_on_db_query_result_update_round_result:95) new unpack [79] ms [280c00043fc20012] [01000c280f0f6315] [01007e2844692235]
roomsvr_40.12.1.log_err: [20221102 15:33:03.256256] [754] <SYS> [0] (room_on_db_query_result_update_round_result:114) old unpack [536] ms [280c00043fc20012] [01000c280f0f6315] [01007e2844692235]
[user00@VM-11-178-centos ~/lsqame/log/dev]$

```

架构的演变 – C/S协议的升级

TDR2是否也适用于C/S的协议?

- 编码开销过大, 性能低, 对战场有较大的影响

C/S协议 – 基于TDR1的扩展-TDREX

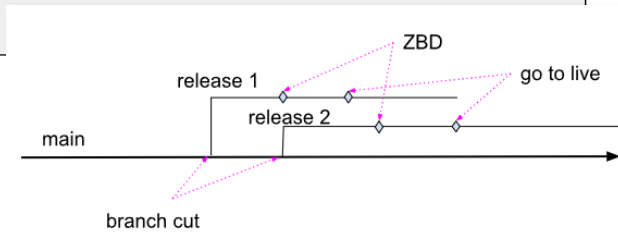
- 自定义的字段删除兼容方案
- 对客户端透明, 无需改造
- 支持协议分段和分支协议合入 – 解决不同分支间的兼容性

字段删除按以下格式:

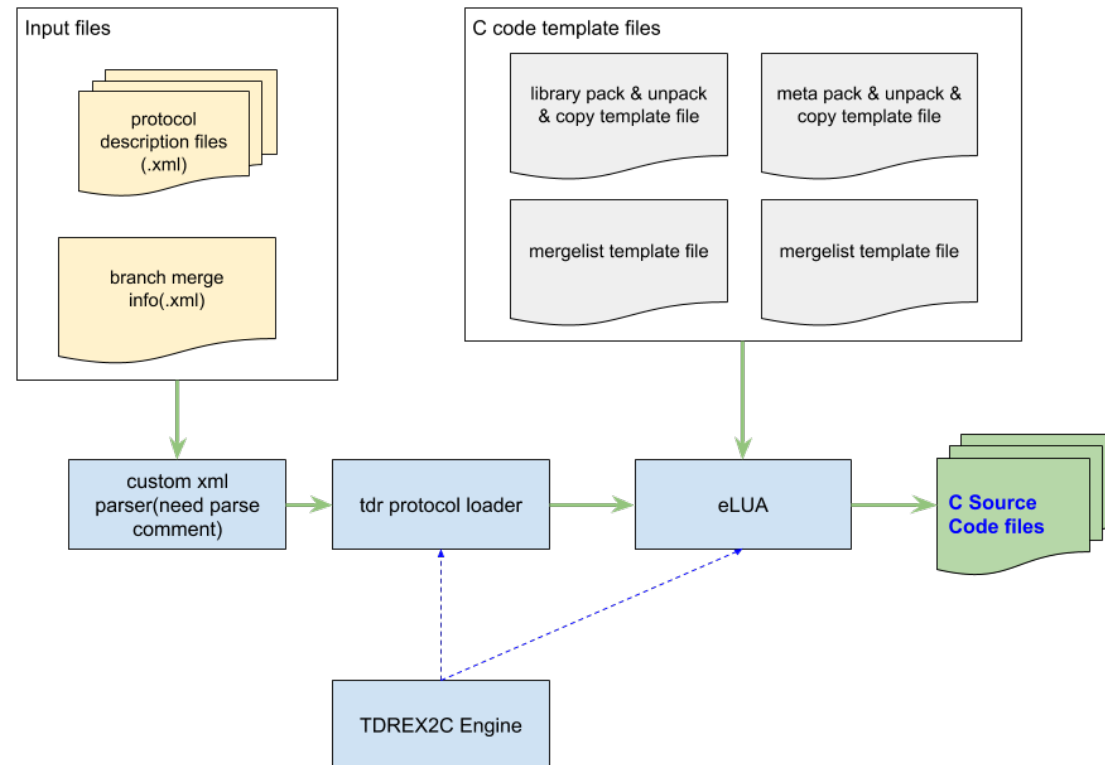
```
<!-- <entry name="FieldName" type="int" version="100" desc="desc of entry" deleteversion="1234"/> -->
```

Union/Struct的整体删除:

```
<!--  
<struct name="TestRut" version="1" desc="desc" deleteversion="1234">  
  <entry name="FieldName" type="int" version="100" desc="desc of entry"/>  
  <.../>  
</struct> -->
```



```
<range from="1309" to="1324" desc="for 1.1">  
  <merge start="1309" end="1309" version="1407" desc="merge version from 1.1 to 2.0"/>  
  <merge start="1310" end="1310" version="1415" desc="merge version from 1.1 to 2.0"/>  
  <merge start="1311" end="1311" version="1387" desc="change 1387 to 1.1, modify mergelist version: 1.1 to 2.0"/>  
  <merge start="1312" end="1312" version="1417" desc="change 1417 to 1.1, modify mergelist version: 1.1 to 2.0"/>  
</range>  
<range from="1401" to="1449" desc="for 2.0 branch">
```



```
/* it has input data and data version include this field. unpack it from input data stream. */  
/* This field is add in version 2937, then merge add into version 3042. */  
if ( 0 != pcszData && ( iVersion == 0 || iVersion >= 3042 || ( iVersion >= 2937 && iVersion <= 3029 ) ) )  
{  
  /* check if the input data length is enough to unpack an string length with type uint32_t. */  
  if( (iDataLen - iParsedLen) < (int)sizeof(uint32_t) )  
  {  
    return tdrex_set_error(TDREX_ERR_OUT_OF_BUFFER, "RoleProperty", "AramNewPlayerScore", -1, "out of buffer while unpack field RoleProperty.AramNewPlayerScore.");  
  }  
  /* parse field value from data stream. */  
  pstRoleProperty->iAramNewPlayerScore = (int32_t)ntoh32(*(uint32_t*)(pcszData + iParsedLen));  
  /* append parsed len to output iParsedLen */  
  iParsedLen += sizeof(uint32_t);  
}  
else  
{  
  /* set the field to default value */  
  pstRoleProperty->iAramNewPlayerScore = (uint32_t)0;  
}
```

当产生分支修改和合入时生成的版本号判断逻辑

架构的演变 – 消息机制

统一消息结构

- 将请求和响应封装在一个结构内
- 服务器不再区分请求端和响应端
- 消息头携带更多路由相关字段

统一消息处理框架

- 将消息的接收和处理进行封装
- 引入消息分发表，将消息ID映射到处理函数
- 可定制的消息前处理

消息通用化

- 通用化的DB请求
- 通用化的各种第三方服务的请求
- 通过携带的子Cmd来区别业务

快速路由转发机制

- 在消息解包之前的快速检测
- 转发消息直接使用原始数据包
- 可选，只需要部分服务器支持

SSMsgC: 请求

Cmd
Cdata: Union

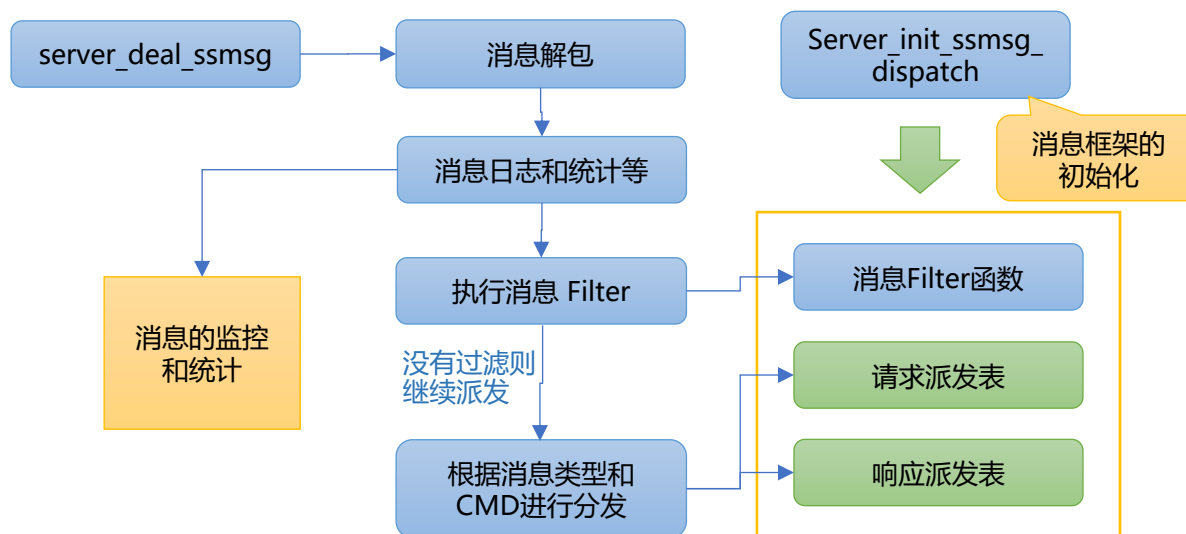
SSMsgS: 响应

Eno
Cmd
SData: Union

SSMsg: 请求/响应

Head: Struct {
Ver,
Uin,
Src,
Target,
Type,
...
}

Body: Union {
SSMsgC,
SSMsgS,
}



架构的演变 – 引入StatusMgr

问题

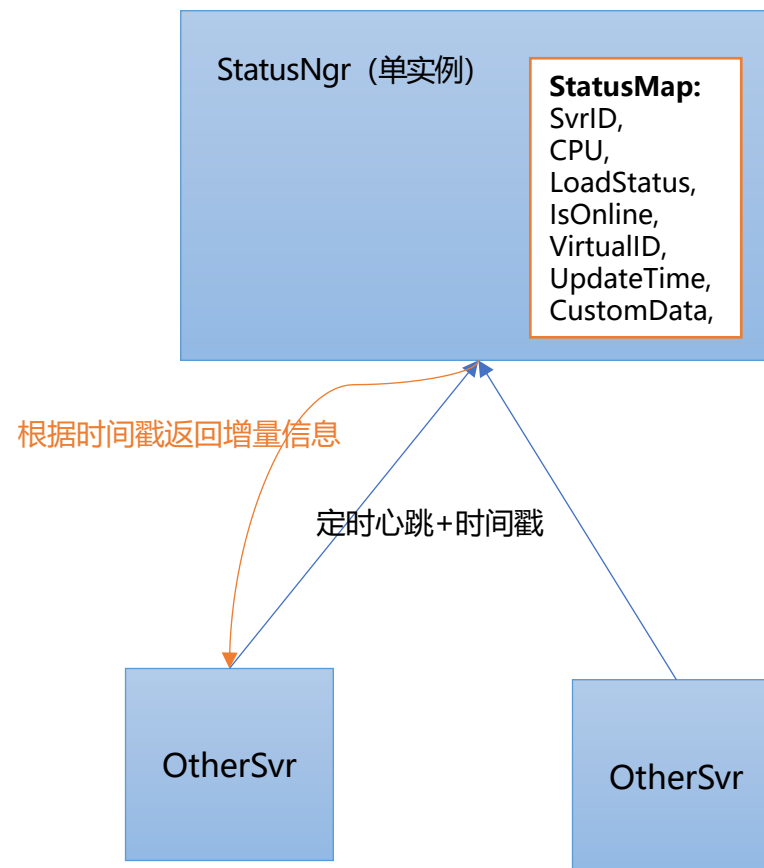
- TBuspp流量分发的策略比较简单，不能满足要求
- 我们需要在服务器间共享一些状态信息
- 跨多集群间的服务状态访问

解决方法 – StatusMgr服务

- 心跳管理
- 服务共享状态同步
- 跨集群的StatusMgr信息广播

应用示例

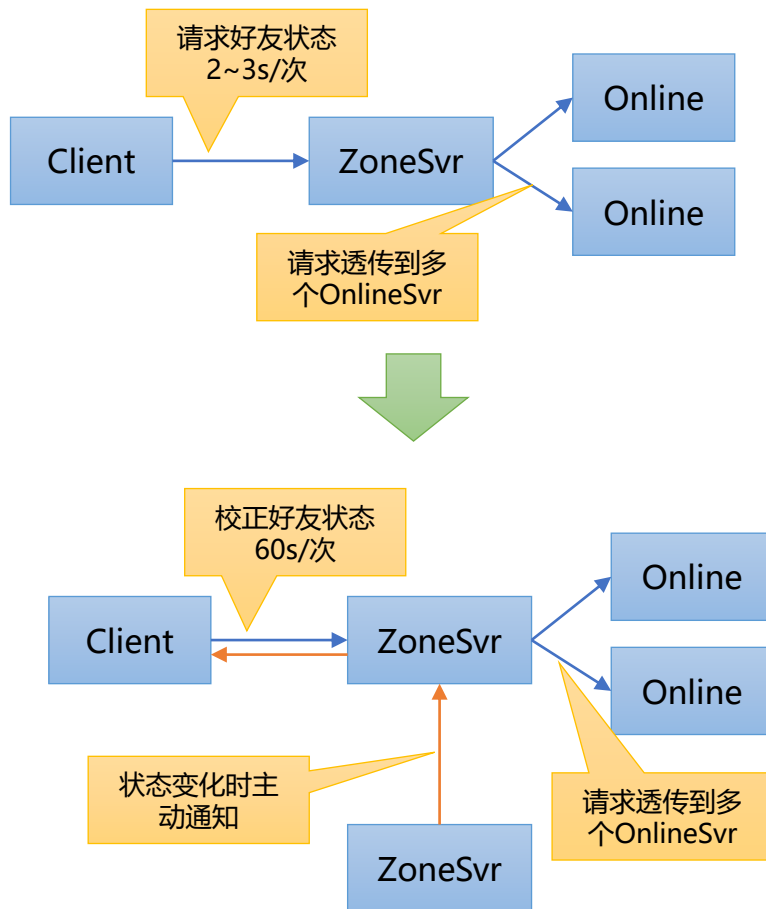
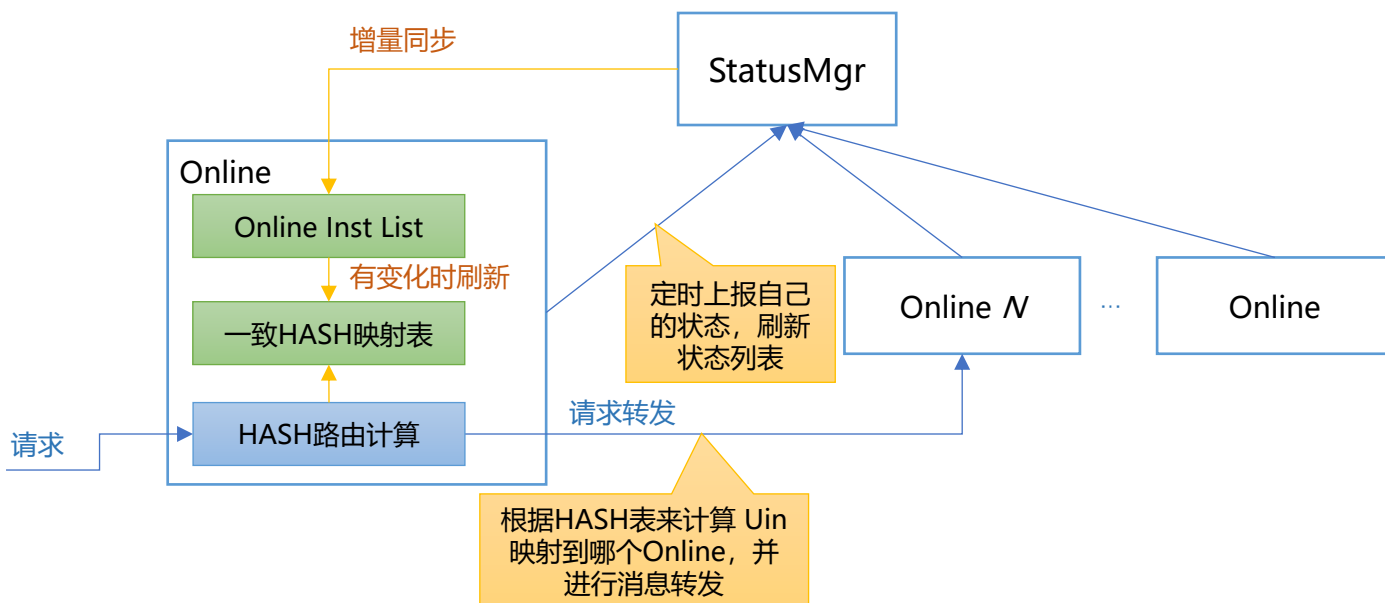
- OnlineSvr的一致性HASH映射关系
- RoomSvr的自定义负载均衡
- 战斗集群可用性探测



架构的演变 – OnlineSvr弱状态化

Onlinesrv的升级

- 将登录态从Online保存改为DB字段 – 达成弱状态
- 通过一致性HASH来进行均衡负载
- 好友状态主动通知优化
- 支持快速转发- 提升性能



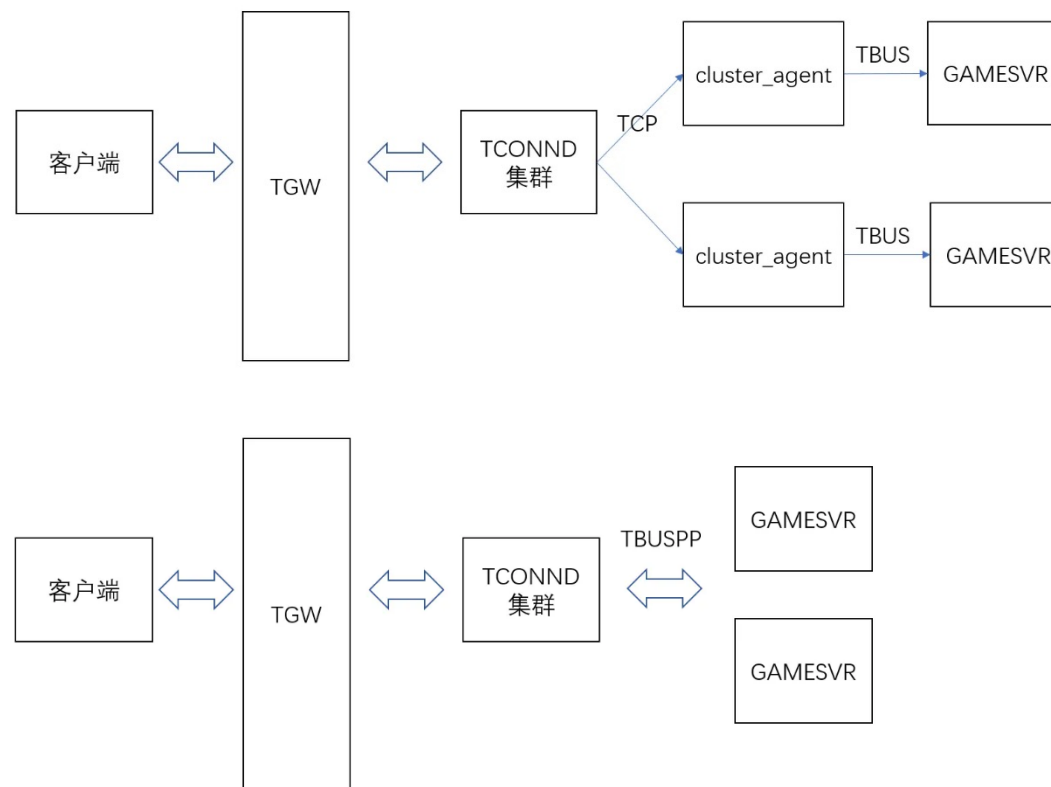
架构的演变 – 其它

Tconnd的集群化

- Tconnd集中部署，可自由扩缩容
- 降低对Tconnd故障的敏感度
- 支持基于客户端的路由保持
- 支持指定Zone登录，可以有更多的可能性。

服务简化和拆分

- WorldSvr – 删除，并将其中的功能独立成，TcapusProxy, ApolloProxy, 等。
- rProxy – 删除，已经不需要进行消息的中转。
- 基于FuncSvr框架分拆增加一些全局功能性的服务模块，如 RoomMgr, MySQLConn, MassConn等。
- 将IDIPServer (原名CSServer) 从Zone级别放到Cluster。
- TcapusProxy, ApolloProxy, 等代理类的服务，进行了业务的剥离，让它更通用化，这样我们在修改，新增业务时，基本不用再修改这些Proxy类的服务了。



核心基础特性设计



国际化方案



通用服务化开发框架



不停服设计



异步开发框架



扩展数据框架



动态数据配置



日志系统的改造



跨服务的GM框架



弱网综合解决方案

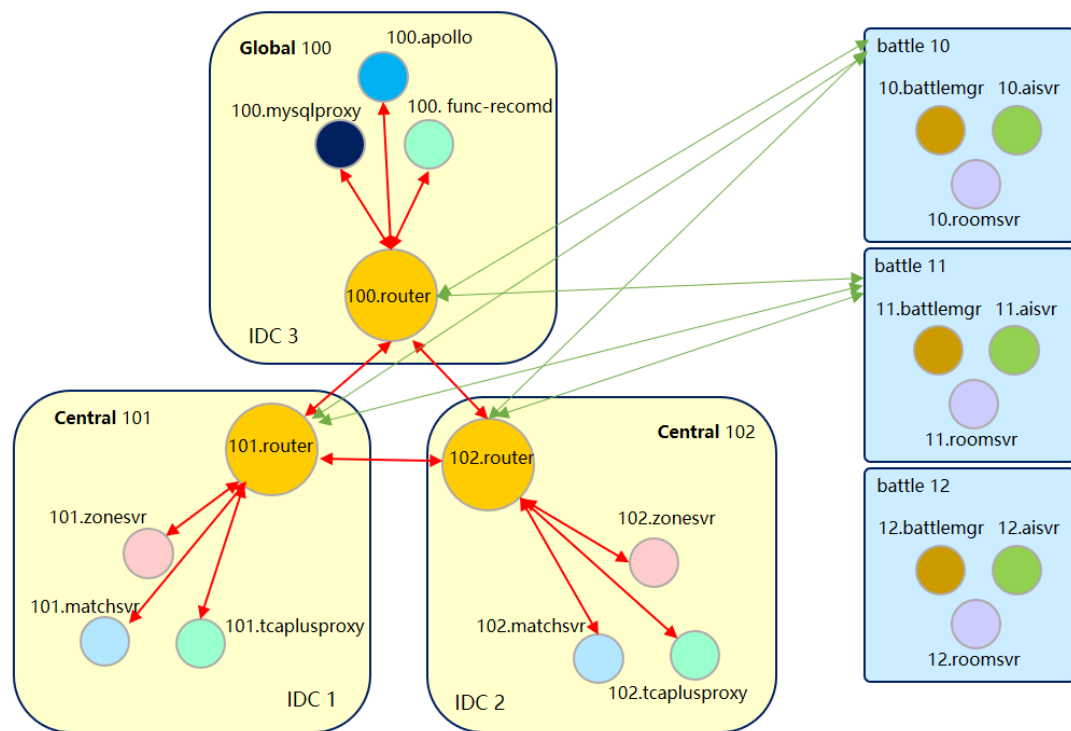


服务器登录安全

国际化方案 - 分区域部署

设计的要求

- 数据合规 – 不同地区有相应的合规要求，如CCPA，GDPR。
- 全球体验一致性 – 全球统一客户端，统一帐号，相同的内容。



Global:

只负责帐号唯一性注册，全局性配置等的处理，即使通讯中断，影响有限。

Central:

负责某个大区的大厅登录，匹配等服务。如欧洲，美洲，中国大陆。延迟要求不高。但承载大。这里负责核心数据的存储和处理。

Battle:

负责就近的战斗服务。低延时。如日本东京，巴西圣保罗机房战斗集群

Global为设计为唯一分区，在实际部署上，因为合规的原因，Amer集群负责的美洲区设计为和其它区(RoW)隔离的形式。所以Amer和RoW是各自拥有自己的Global分区。

国际化方案 – 分区间Router方式

Router的必要性:

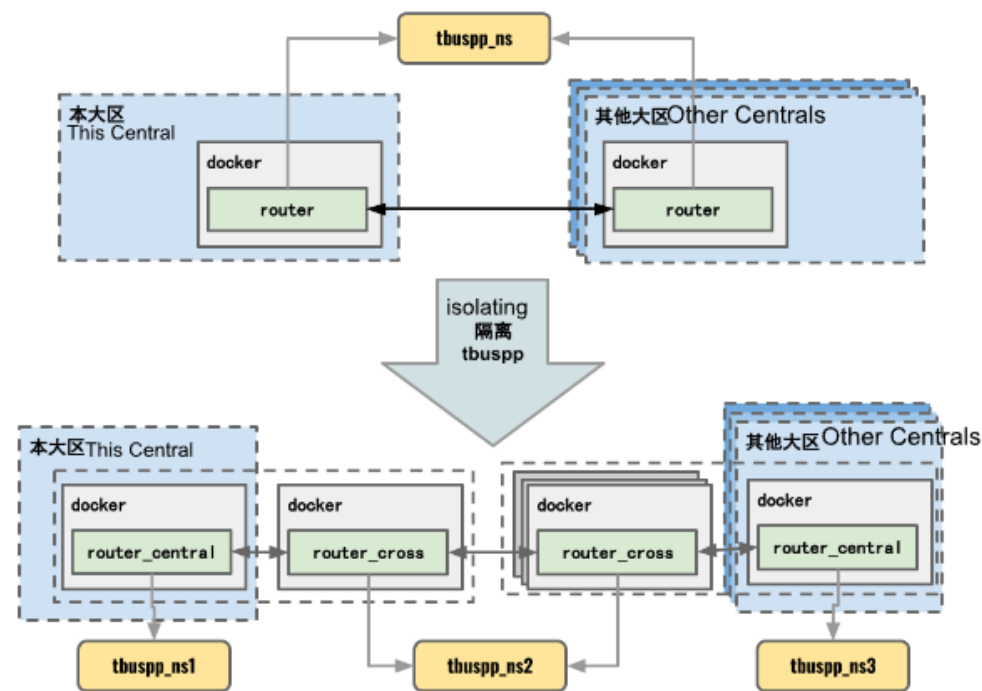
- 分区往往不在同一机房，甚至相离很远。集群间需要通过VPN等方式互联。
- Tbuspp名字服务在跨洲网络下有性能风险。需要分区部署。
- 服务之间直接发送消息会有一定的风险。

通过Router收敛了集群间的流量，可以进行更好的流量优化，消息加密等。同时也方便了对集群间流量进行监控。

需要解决的问题:

- 多个Tbuspp名字空间之间节点如何互联

双容器Router方案:



两个容器共同组成一个Pod。分为内/外两个节点，内节点使用集群内的空间负责和集群内部节点通讯，外节点共享同一个名字空间，负责集群间的互联。内外节点间使用Tbus进行单机内部通讯。

国际化方案 – 区域定制化

代码层定制化:

- ◆ 登录方式 (Riot VS. WX/QQ)
- ◆ 关系链 (Riot关系链 VS. WX/QQ)
- ◆ 敏感词检测 (CN独有)
- ◆ Riot帐号ID映射 (ROW独有)

方案: 通过宏开关编译为不同的二进制版本。

环境定制化:

- ◆ 环境的连接参数
- ◆ 环境变量, 进程配置
- ◆ 分区进程定制
- ◆ 应用模板区域定制

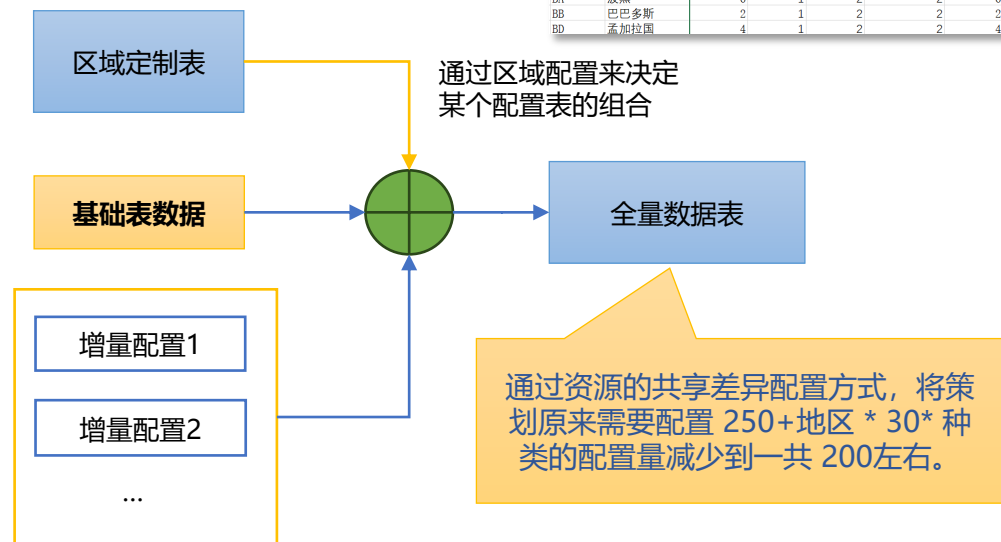
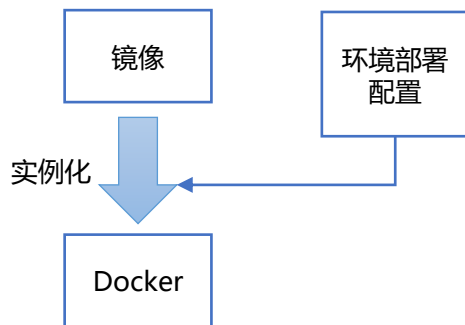
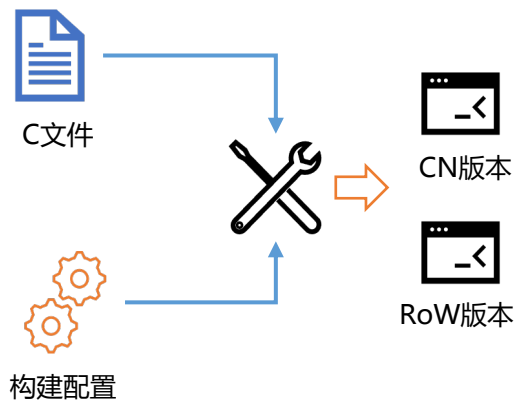
方案: 分地区的部署脚本, ConfigMap, 应用模板配置等。

资源配置定制化:

- ◆ 策划的配置表格
- ◆ 运营活动
- ◆ 网络策略
- ◆ 匹配参数

方案: 策划表格 – 按功能分级, 地区差异增量配置。如果多个地区的配置相同, 则共享差异。

地区代码	地区中文名称	商城分组	登录分组	背包分组	局内设置分组	活动分组	新手
AD	安道尔	6	1	2	2	6	
AE	阿联酋	4	1	2	2	4	
AF	阿富汗	4	1	2	2	4	
AG	安提瓜和巴布达	2	1	2	2	2	
AI	安圭拉	2	1	2	2	2	
AL	阿尔巴尼亚	6	1	2	2	6	
AM	亚美尼亚	10	5	2	2	10	
AO	安哥拉	6	1	2	2	6	
AQ	南极洲	6	1	2	2	6	
AR	阿根廷	3	1	2	2	3	
AS	美属萨摩亚	2	1	2	2	2	
AT	奥地利	6	1	2	2	6	
AU	澳大利亚	3	1	2	2	3	
AW	阿鲁巴	3	1	2	2	3	
AX	奥兰	6	1	2	2	6	
AZ	阿塞拜疆	10	5	2	2	10	
BA	波斯尼亚和黑塞哥维那	6	1	2	2	6	
BB	巴巴多斯	2	1	2	2	2	
BD	孟加拉国	4	1	2	2	4	



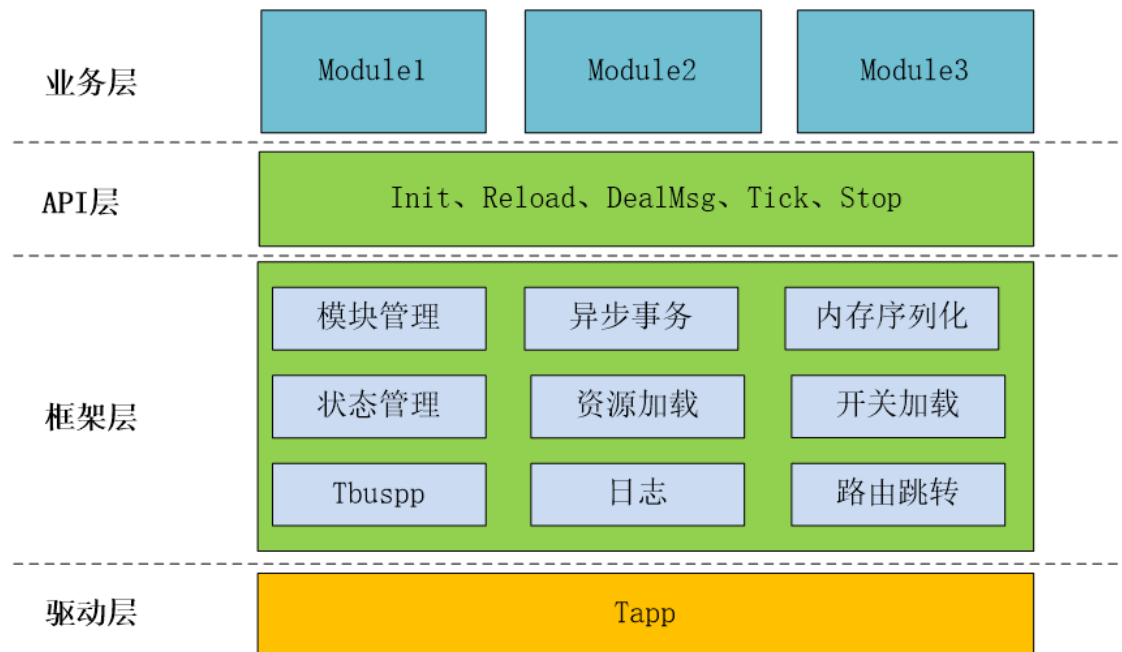
通用服务化开发架构 – FuncSvr框架

为什么要做通用服务框架？

- 微服务的理念下，各功能倾向细分。单一服务尽可能只做一件事。
- 各服务对于很多通用的基础功能都有相同的诉求。
- 如何平衡单一功能服务和服务数量的增长给开发过程带来的挑战。

FuncSvr的主要特点：

- 封装业务都需要的通用功能。
- 将业务功能抽象成一个个模块，提供统一的管理，加载，调用方式。
- 封装对业务模块的统一消息路由，分发等机制。
- 灵活的部署方式 – 支持单进程多模块，通过配置来控制。



```
// macros
struct tagFuncsvrModule
{
    int iModuleID; //模式ID
    const char szName[RES_MAX_64_LEN]; //模式名称
    ModuleFunc1 pfnInit;
    ModuleFunc1 pfnTick;
    ModuleFunc1 pfnStop;
    ModuleFunc1 pfnReload;
    ModuleFunc2 pfnDealMsg;
    ModuleFunc3 pfnGetNeedTable;
    ModuleFunc4 pfnGetNeedShm;
    ModuleFunc1 pfnRetLoad;

    char chIsOpen;
};
typedef struct tagFuncsvrModule FUNC_SVRMODULE;
typedef struct tagFuncsvrModule* LP_FUNC_SVRMODULE;
```

```
// global variables
FUNC_SVRMODULELIST gs_stFuncsvrModuleList =
{
    //这个数字也要必须得一致
    18,
    {
        //recomd模块,模块名不要有空格
        { FUNC_FUNC_SVR_TEST, "test", test_init, test_tick, test_stop, NULL, test_dealmsg, test_need_table, test_need_shm, NULL, 0 },
        { FUNC_FUNC_SVR_RECOMD, "recomd", recomd_init, recomd_tick, recomd_stop, NULL, recomd_dealmsg, recomd_need_table, recomd_need_shm, recomd_ret_load, 0 },
        { FUNC_FUNC_SVR_MATCHROOM, "matchroom", matchroom_init, matchroom_tick, matchroom_stop, matchroom_reload, matchroom_dealmsg, matchroom_need_table, matchroom_need_shm, matchroom_ret_load, 0 },
        { FUNC_FUNC_SVR_TEAM, "team", team_init, team_tick, team_stop, team_reload, team_dealmsg, team_need_table, team_need_shm, team_ret_load, 0 },
        { FUNC_FUNC_SVR_MYSQL, "mysql", module_mysql_init, module_mysql_tick, module_mysql_stop, NULL, module_mysql_dealmsg, module_mysql_need_table, module_mysql_need_shm, module_mysql_ret_load, 0 },
        { FUNC_FUNC_SVR_MASS, "mass", mass_init, mass_tick, mass_stop, NULL, mass_dealmsg, mass_need_table, mass_need_shm, mass_ret_load, 0 },
        { FUNC_FUNC_SVR_TCAPLUSPROXY, "tcaplusproxy", tcaplusproxy_init, tcaplusproxy_tick, tcaplusproxy_stop, tcaplusproxy_reload, tcaplusproxy_dealmsg, tcaplusproxy_need_table, tcaplusproxy_need_shm, tcaplusproxy_ret_load, 0 },
        { FUNC_FUNC_SVR_STATUSMGR, "statusmgr", statusmgr_init, statusmgr_tick, statusmgr_stop, statusmgr_reload, statusmgr_dealmsg, statusmgr_need_table, statusmgr_need_shm, statusmgr_ret_load, 0 },
        { FUNC_FUNC_SVR_APOLLO, "apollo", apollo_init, apollo_tick, apollo_stop, apollo_reload, apollo_dealmsg, apollo_need_table, apollo_need_shm, apollo_ret_load, 0 },
        { FUNC_FUNC_SVR_ANTIICON, "antiicon", antiicon_init, antiicon_tick, antiicon_stop, antiicon_reload, antiicon_dealmsg, antiicon_need_table, antiicon_need_shm, antiicon_ret_load, 0 },
        { FUNC_FUNC_SVR_FACEDTECT, "facedetect", facedetect_init, facedetect_tick, facedetect_stop, facedetect_reload, facedetect_dealmsg, facedetect_need_table, facedetect_need_shm, facedetect_ret_load, 0 },
        { FUNC_FUNC_SVR_GUILD, "guild", guild_init, guild_tick, guild_stop, guild_reload, guild_dealmsg, guild_need_table, guild_need_shm, guild_ret_load, 0 },
        { FUNC_FUNC_SVR_LOBBYCHATV3, "lobbychatv3", lobbychatv3_init, lobbychatv3_tick, lobbychatv3_stop, lobbychatv3_reload, lobbychatv3_dealmsg, lobbychatv3_need_table, lobbychatv3_need_shm, lobbychatv3_ret_load, 0 },
        { FUNC_FUNC_SVR_LOBBYCHATMGRV3, "lobbychatmgrv3", lobbychatmgrv3_init, lobbychatmgrv3_tick, lobbychatmgrv3_stop, lobbychatmgrv3_reload, lobbychatmgrv3_dealmsg, lobbychatmgrv3_need_table, lobbychatmgrv3_need_shm, lobbychatmgrv3_ret_load, 0 },
        { FUNC_FUNC_SVR_RUNEEQUIP, "runeequip", runeequip_init, runeequip_tick, runeequip_stop, runeequip_reload, runeequip_dealmsg, runeequip_need_table, runeequip_need_shm, runeequip_ret_load, 0 },
        { FUNC_FUNC_SVR_GAMECENTER, "gamecenter", gamecenter_init, gamecenter_tick, gamecenter_stop, gamecenter_reload, gamecenter_dealmsg, gamecenter_need_table, gamecenter_need_shm, gamecenter_ret_load, 0 },
        { FUNC_FUNC_SVR_SYNCROSS, "syncross", syncross_init, syncross_tick, syncross_stop, syncross_reload, syncross_dealmsg, syncross_need_table, syncross_need_shm, syncross_ret_load, 0 },
        { FUNC_FUNC_SVR_SIMULATEAI, "simulateai", simulateai_init, simulateai_tick, simulateai_stop, simulateai_reload, simulateai_dealmsg, simulateai_need_table, simulateai_need_shm, simulateai_ret_load, 0 },
        { FUNC_FUNC_SVR_RECOVERYROOMMGR, "roomrecoverymgr", roomrecoverymgr_init, roomrecoverymgr_tick, roomrecoverymgr_stop, roomrecoverymgr_reload, roomrecoverymgr_dealmsg, roomrecoverymgr_need_table, roomrecoverymgr_need_shm, roomrecoverymgr_ret_load, 0 },
    }
};
```


通用服务化开发架构 – FuncSvr模块配置和路由

- ◆ 通过数组和Union的结合，将所有模块的配置组合在一起：

```
<struct name="ModuleCfg" version="1" desc="模块配置">
  <entry name="ModuleName" type="string" size="256" desc="模块名称" />
  <entry name="ModuleID" type="int" desc="模块ID,模块名称, 其实目前和FuncID是一致的"/>
  <entry name="ModuleSelfConfig" type="ModuleSelfConfig" desc="模块特有的配置" select="ModuleID"/>
</struct>

<union name="ModuleSelfConfig" version="1" desc="模块特有的配置">
  <entry id="FUNC_FUNC_SVR_MYSQL" name="ModuleMysql" type="MysqlList" desc="Mysql的配置文件"/>
  <entry id="FUNC_FUNC_SVR_MATCHROOM" name="ModuleMatchRoom" type="ModuleMatchRoomCfg"/>
  <entry id="FUNC_FUNC_SVR_MASS" name="ModuleMass" type="MassCfg" desc="防沉迷配置"/>
  <entry id="FUNC_FUNC_SVR_TCAPLUSPROXY" name="ModuleTcaplusProxy" type="TcaplusProxyCfg" desc="TcaplusProxy配置"/>
  <entry id="FUNC_FUNC_SVR_APOLLO" name="ModuleApollo" type="ModuleApolloCfg" desc="Apollo配置"/>
  ...
</union>

<struct name="FuncsvrSysConfig" version="1">
  ...
  <entry name="ServiceName" type="int" desc="服务名称, 填FuncID就可以了" bindmacrosgroup="ENUM_FUNC"/>
  <entry name="ModuleNum" type="int" desc="启动的模块数量"/>
  <entry name="ModuleList" type="ModuleCfg" count="MAX_FUNC_SVR_MODULE_NUM" refer="ModuleNum" desc="启动的模块列表"/>
  ...
</struct>
```

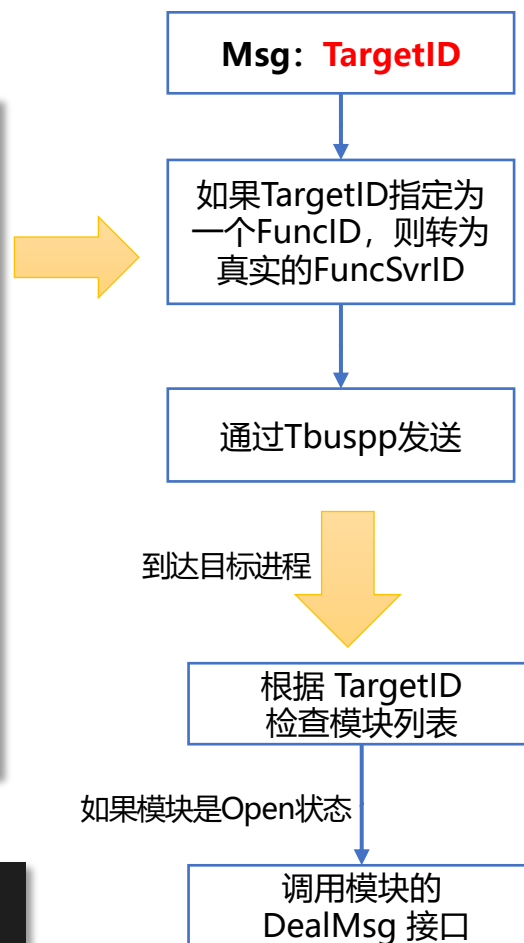
- ◆ 通过进程配置文件 ProcDeploment 来指定哪些模块会在哪些 FuncSvr 进程上启动：

```
{name = "funcsvr_statusmgr", index = 1, modules = {FUNC_FUNC_SVR_STATUSMGR, FUNC_FUNC_SVR_TCAPLUSPROXY}},
{name = "funcsvr_recomd", index = 1, modules = {FUNC_FUNC_SVR_RECOMD, FUNC_FUNC_SVR_MATCHROOM, FUNC_FUNC_SVR_TEAM, FUNC_FUNC_SVR_TEAMRECRUIT}},
{name = "funcsvr_recomd", index = 2, modules = {FUNC_FUNC_SVR_RECOMD, FUNC_FUNC_SVR_MATCHROOM, FUNC_FUNC_SVR_TEAM, FUNC_FUNC_SVR_TEAMRECRUIT}},
{
```

- ◆ 当进程实例化时，根据配置来生成对应的FuncSvr配置及其上的模块配置列表。同时会生成一个模块路由表。

模块路由表：

```
<ResFuncsvrRoute version="1">
  <SrcFuncID>113 </SrcFuncID>
  <DstFuncID>113 </DstFuncID>
  <Desc>funcsvr-recomd </Desc>
</ResFuncsvrRoute>
<ResFuncsvrRoute version="1">
  <SrcFuncID>114 </SrcFuncID>
  <DstFuncID>113 </DstFuncID>
  <Desc>funcsvr-matchroom </Desc>
</ResFuncsvrRoute>
<ResFuncsvrRoute version="1">
  <SrcFuncID>115 </SrcFuncID>
  <DstFuncID>113 </DstFuncID>
  <Desc>funcsvr-team </Desc>
</ResFuncsvrRoute>
<ResFuncsvrRoute version="1">
  <SrcFuncID>129 </SrcFuncID>
  <DstFuncID>113 </DstFuncID>
  <Desc>funcsvr-guild </Desc>
</ResFuncsvrRoute>
```



不停服改造 – 总体设计

痛点:

在大版本更新时需要停服4~8个小时

- 玩家长时间不能登录导致流失。
- 停服更新一般需要在凌晨操作以减少对玩家的影响，但对研发团队体验较差。
- 停服更新的工作量大，发布效率低。
- 如果更新出现问题不能按时开服，会影响口碑。

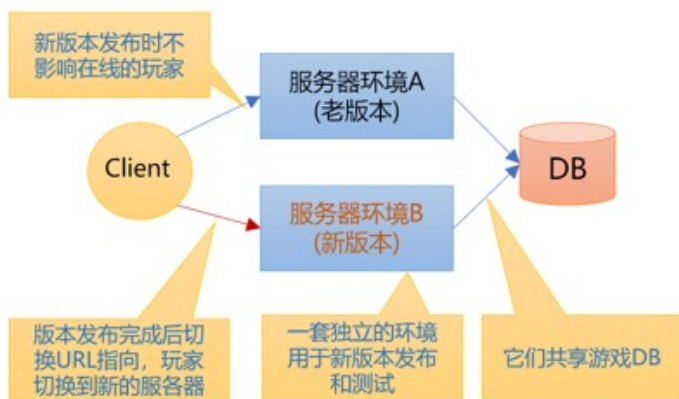
方案一：双环境切换

优点：

- 方案简单

不足：

- 双环境成本高
- 运维发布过程复杂
- 游戏中的玩家不能立刻切换，需要特殊处理。
- 切换过程体验不佳



综合考虑：实际意义不大，放弃。



对于游戏业务这种相对重度的服务器，能否做到大版本发布时的不停服更新？

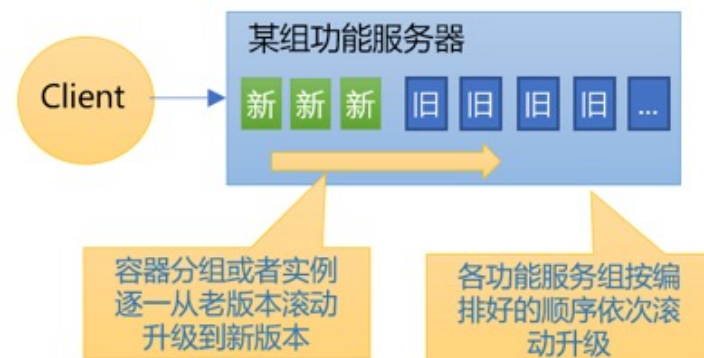
方案二：滚动更新

优点：

- 成本低
- 借助容器滚动机制
- 自动化程度高
- 中间有问题可以随时暂停进行处理
- 玩家基本无感

不足：

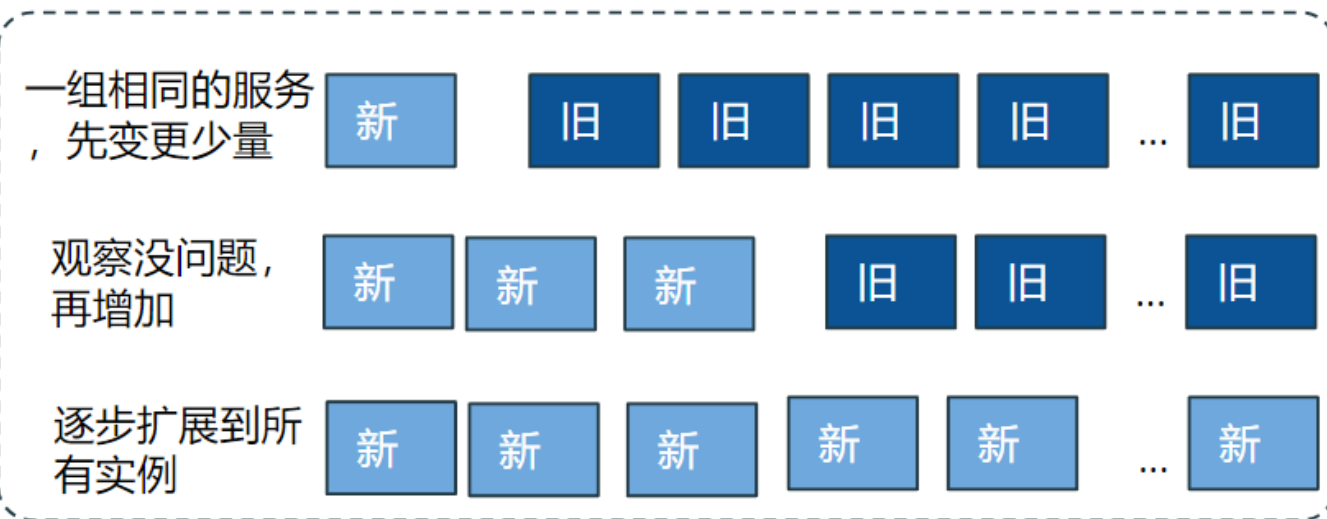
- 滚动过程考虑兼容性
- 战场导流机制



采用此方案进行进一步细化设计。

不停服改造- 灰度切换的设计

服务类型	灰度方式	举例	效果
无状态的服务	直接重启，无特殊操作	TacplusProxy, ApolloProxy,	无感
弱状态 - 可丢弃	重启前清理，如取消匹配	MatchSvr, TeamSvr	玩家需要重新操作，有少量体验损失
弱状态 - 可重建	重启后重建或者访问时重建，如Online重启后会向大厅通知重建玩家状态	Online, 各类Cache服务	玩家基本无感，或者少量操作延时
强状态 - 重连触发加载	ZoneSvr, 退出前会保存玩家状态并断线，玩家静默重连后重新建立内存	ZoneSvr, Guild, Chat	玩家基本无感，或者少量操作延时
强状态 - 内存主动恢复	服务退出时数据回写DB，服务启动后主动从DB加载完全重建内存	CustomRoom	玩家无感，或者少量操作延时



服务优雅退出机制

解决退出时未完成请求丢失问题

- ❑ **Tbuspp预退出** - 不再的新的请求，但继续处理回包。延时一时间 后，确认没有新的回包再真实退出。
- ❑ **StatusMgr预退出** - 自定义负载均衡服务，通过设置StatusMgr上和退退出标识，不再负载分配。
- ❑ **Tconnd过载保护** - 退出前，设置过载状态，不再分配新的用户。等待老用户保存完成并下线。

不停服改造- 战场导流

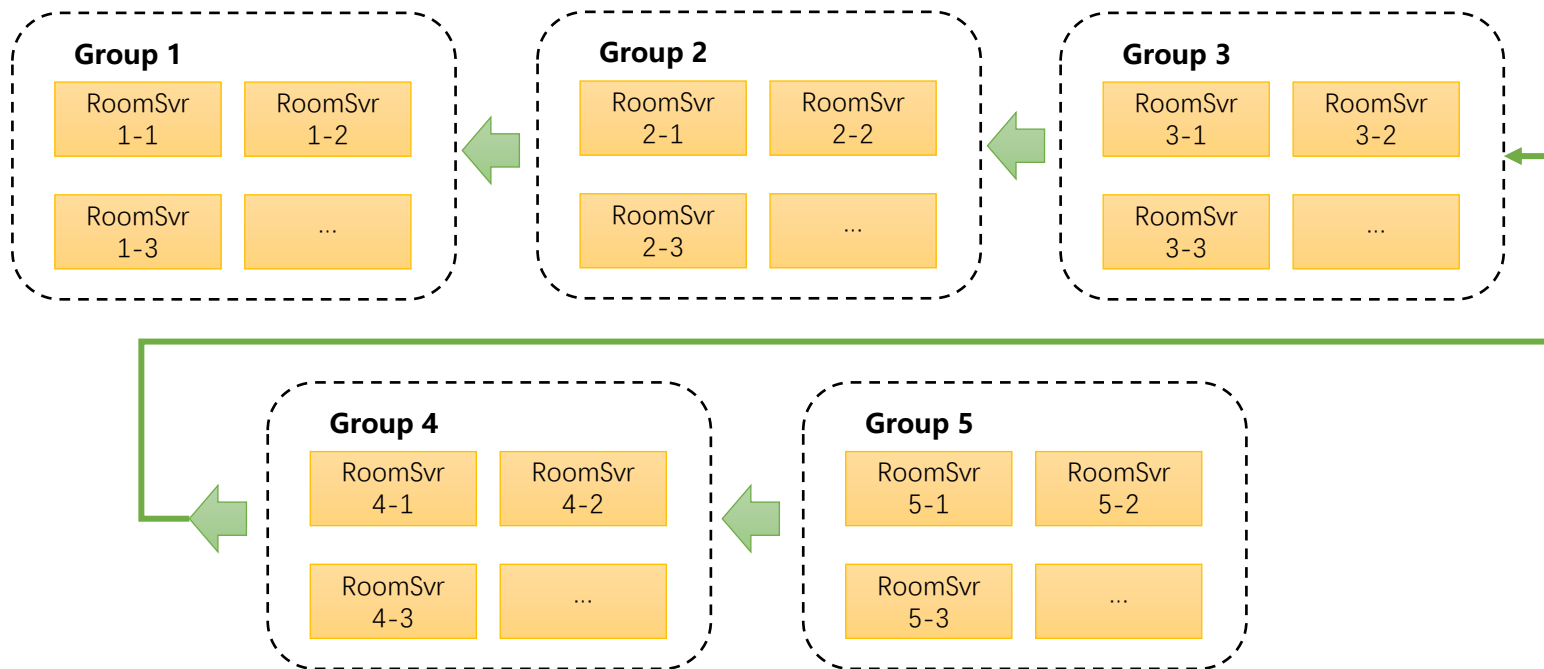
RoomSvr的特点:

- ✓一局时间长（20分钟左右），一局未结束前不能中断。
- ✓RoomSvr 数量多。如果依次滚动升级，耗时过长。

解决思路:

- ✓分组，先导流再变更。
- ✓根据战局的最大时长，需要设置一个较长的停服超时时间。
- ✓为了加速变更过程，RoomSvr的分配是优先使用满(80%)一组再分配下一组。

分组:



导流过程:

1. 打开容器组的导流配置

下发配置，打开导流后将不会再分配新的战局到这一组RoomSvr上。

2. 等待已有的战斗结束

等待RoomSvr上正在运行的所有战局都结束后

3. 执行滚动变更

对该组容器执行滚动变更，把所有的容器都替换为新的版本

不停服改造- 通用内存状态保存恢复

为什么需要内存状态保存?

- 不停服更新时需要进行有状态服务数据的恢复
- 当服务异常退出时, 有状态服务可能会丢失数据
- 各服务状态数据各不相同, 但基本的保存和恢复逻辑是可以通用的

SHM可序列化设计

数据SHM的 Key- Value 设计

使用TDR的数据结构描述

将Value映射到一系列桶中

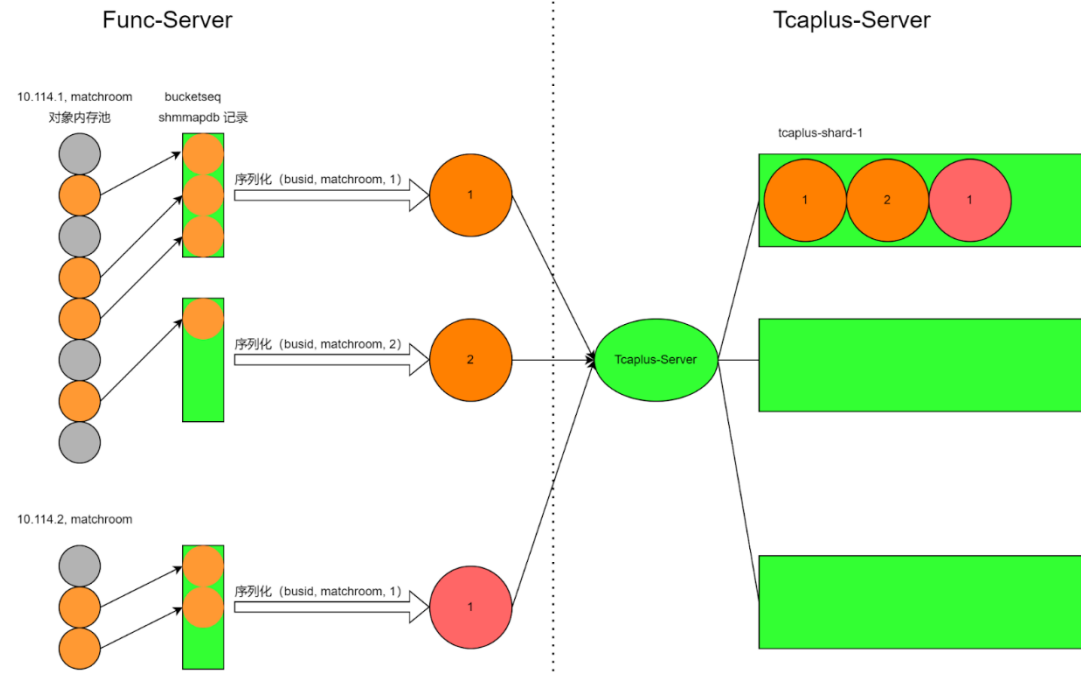
数据修改版本号机制

数据保存的时机:

- Tick触发, 脏数据回写
- 重要数据修改时, 业务主动保存

数据恢复的时机:

- 服务启动时, 从DB读取相应 BusID的记录, 解析出每个Value插入到内存。
- 可通过配置来关闭数据恢复过程。



SHM2DB表的结构:

字段	说明	类型
Busid	所属svr的 busid	Int32
Shm	内存池名称	字符串
VKey	数据的key	Int32
DataLen	数据长度	Int32
Data	数据buff, 250K	二进制数据

不停服改造-兼容性设计

升级过程的兼容性考虑

• 资源兼容性:

- 通过版本号来区分新版本的内容。如新的活动，任务，道具。
- 需要持久化的资源，理论上不应该删除，或者需要有清理逻辑。

• 协议兼容性:

- S/S之间的TDR2.0可以确保在升级过程中保持消息兼容（需要注意升级顺序，服务可能有功能上的依赖顺序，以及协议的规范设计）。

• C/S之间协议的兼容性:

- 由服务器进行协议裁剪（根据客户端的版本号），注意，不能立刻删除一个客户端需要的协议字段。应该保持兼容一段时间。

• 逻辑兼容性:

- 如果新版本逻辑变动，要考虑老客户端是否能正常工作。必要的情况下，应该在一段时间内保持两种逻辑工作，直到版本完成升级。

• 数据兼容性:

- 当涉及到客户端的数据结构发生变化，需要通过客户端的版本号来控制数据的变更，进到客户端升级

工具扫描协议改动点:

协议文件	main协议	owner	owner填写关注模块	owner确认CS协议是否下发该字段	owner确认客户端使用该字段时，但旧版本解包该字段为空或默认值，是否影响功能	owner确认是否兼容
star_comm.xml	<!-- <entry id="98" deleteversion="4241" name="HexDailyFirstFinishPlus" type="int" defaultvalue="0" version="3083" desc="海克斯能量每日首局加成"/> -->	en	海克斯结算	是	无加成正常	是
star_comm.xml	<!-- <entry id="97" deleteversion="4230" name="HistoryIniSkillItem" type="HistoryIniSkillItem" desc="历史魔典技能数量" version="4025"/> -->	en	终极魔典技能	否		是
star_comm.xml	<!-- <entry id="12" deleteversion="4208" name="HeroStat" type="SeasonHisHeroList" version="4086" desc="英雄数据统计"/> -->	ui	无	否	不影响	是
star_shm2db.xml	<!-- <entry id="113" deleteversion="4230" name="HistoryIniSkillItem" type="HistoryIniSkillItem" desc="历史魔典技能数量" version="4025"/> -->	en	终极魔典技能	否		是
star_shm2db.xml	<!-- <entry id="124" name="ExtRoleForbidCsCmdData" deleteversion="4185" type="ExtRoleForbidCsCmdData" desc="禁止CS命令" version="4067"/> -->	uild	禁止CS命令	否		是
star_ss.xml	<!-- <entry id="35" deleteversion="4230" name="HistoryIniSkillItem" type="HistoryIniSkillItem" desc="历史魔典技能数量" version="4025"/> -->	en	终极魔典技能	否		是

协议文件	4.1.0协议	main协议	owner	owner填写关注模块	owner确认4.1.0和4.2.0版本svr相互通信是否兼容	owner确认4.1.0和4.2.0版本svr相互通信是否兼容
star_aisvr.xml	<macro name="AISVR_MAX_MSG_LEN" value="61000" desc="最大的消息长度，UDP包太长会分包重组，增加丢包的概率，最好不要超过500字节"/>	<macro name="AISVR_MAX_MSG_LEN" value="63000" desc="最大的消息长度，UDP包太长会分包重组，增加丢包的概率，最好不要超过500字节"/>	an	扩大服务化协议打包长度；	兼容	兼容
star_comm.xml	<macro name="LOG_SHM_TYPE_MAX" value="10" desc="注：最大类型，这个就规定了每个进程可以配的最大日志共享内存数目"/>	<macro name="LOG_SHM_TYPE_MAX" value="11" desc="注：最大类型，这个就规定了每个进程可以配的最大日志共享内存数目"/>	in		兼容	兼容
star_def.xml	<macro name="MAX_SHM_MAP_DB_DATA_LEN" value="250000" desc="最大数据大小250K"/>	<macro name="MAX_SHM_MAP_DB_DATA_LEN" value="600000" desc="最大数据大小600K"/>	ou	matchroom内存恢复	兼容	兼容
star_def.xml	<macro name="MAX_EXT_DATA_TYPE_NUM" value="62" desc="最大的存储类型个数"/>	<macro name="MAX_EXT_DATA_TYPE_NUM" value="64" desc="最大的存储类型个数"/>		周报英雄统计数据、周报荣誉数据	兼容	兼容
star_def.xml	<macro name="CHANGE_OBJECT_STEP_MAX" value="9" desc="转区步骤上限"/>	<macro name="CHANGE_OBJECT_STEP_MAX" value="11" desc="转区步骤上限"/>	tg	转区模块	兼容	兼容
star_macro.xml	<macro name="MAX_DISGUISE_ITEM_NUM" value="1999" desc="玩家解锁的装扮道具最大数量"/>	<macro name="MAX_DISGUISE_ITEM_NUM" value="2999" desc="玩家解锁的装扮道具最大数量"/>	u	特效道具模块	兼容	兼容
star_macro.xml	<macro name="MODE_SEASON_HERO_SC_ID_32_MAX" value="6" desc="int32存储最大数量，以后扩展时需要更改"/>	<macro name="MODE_SEASON_HERO_SC_ID_32_MAX" value="10" desc="int32存储最大数量，以后扩展的需要更改"/>	n	周报	兼容	兼容

* 每个版本需要通过工具来扫描可能兼容性问题并一一确认。保证不把兼容性问题带到外网

不停服改造- 总结



总结：

- 微服务和扁平架构的改造，为不停服的设计打下基础。
- 名字服务的引入，Tbuspp的底层通讯为服务上云做好了准备。
- 通过对服务器的无状态化，内存恢复机制的引入，可以让服务支持在线的灰度升级。
- 消息的封装，协议的改造等更进一步增强了服务之间的兼容性。
- 通蓝鲸平台的进程管理，构建和发布流水线的应用，大大提升了系统的可运维性。

收益：

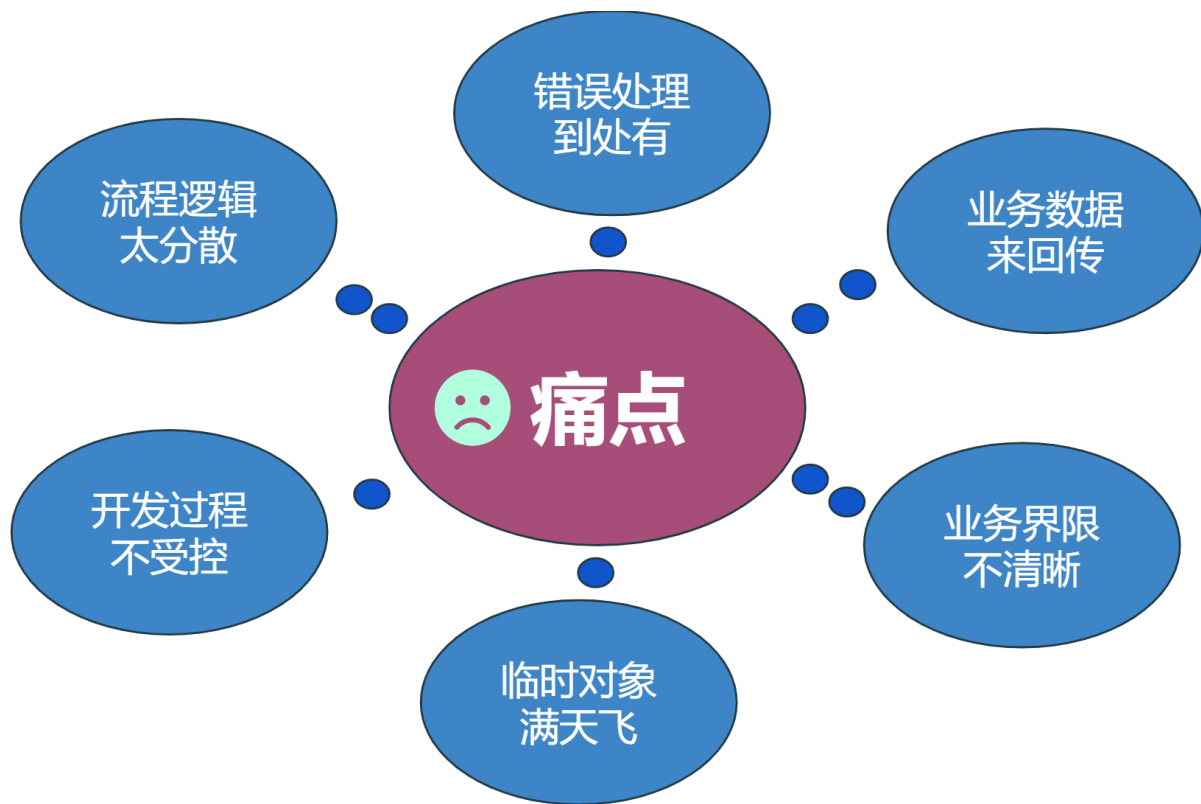
- ✓ 服务器升级期间，玩家基本无感，可以正常进程游戏。
- ✓ 玩家可以自由选择合适的时间下载和升级到新的客户端。
- ✓ 大版本更新一般安排在工作日上午进行，提升了团队的体验，降低出错的可能，并可以更快速的响应和处理异常情况。
- ✓ 版本发布效率更高了，可以在滚动过程中动态检查新服务运行状态，有问题立刻处理。当只有少量的节点更新后发现问题，完全可以先屏蔽这些新版本的节点。

分布式系统的特点

- 服务分布式，多进程，多机器，多地区
- 服务多实例，高并发，容灾
- 业务链路长，异常情况复杂

传统消息系统存在的问题：

- 业务的处理流程不连续，逻辑分散在各个消息处理中
- 业务都有自己的上下文信息，需要在消息中来回传递
- 相同的消息可能被多个业务共享，逻辑混杂在一起
- 异常情况多，错误处理分散在业务流程中
- 为了实现业务之间的关联，需要存储各种临时数据



异步开发框架 – 方案选型



方案：协程？还是RPC？在C语言的基础上，都不是最好的解决方案

我们能否基于现有的基础上，**适用于C语言+消息通讯的机制来实现一套方便易用的基础分布式远程调用框架？**
它需要拥有以下特点：

代码高度整合

单一业务代码可以集中在一起进行管理，方便阅读和维护升级。

业务逻辑复用

支持业务调用已经写好的子业务，做到业务代码复用。

与现有代码兼容

对现有代码的修改小，方便老代码逐步的移植，减少开发工作量。

统一错误处理

业务的执行过程中产生的错误，可以在统一的地方方便地处理。

支持业务上下文

方便地管理和访问业务相关的上下文数据，减少不必要的数据来回传递。

统一远程调用

可以方便调用另一个进程上的服务，并处理返回值。

开发高效安全

通过框架的封装，保证业务在开发和运行的过程中不会出现低级错误。且上手需要很容易。

支持业务监控

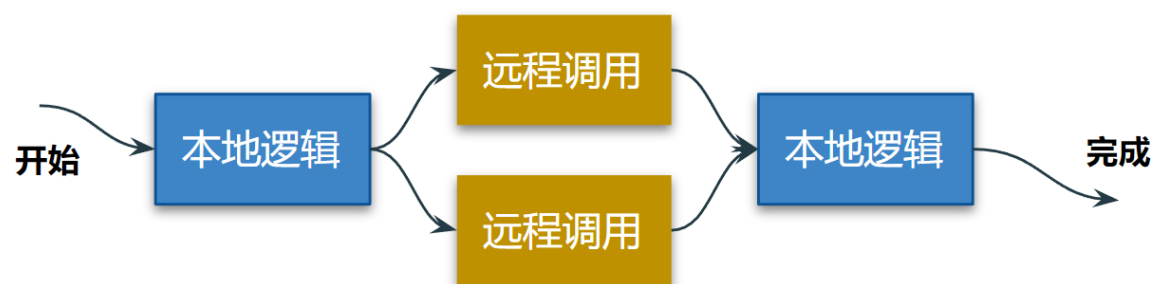
可以监控和采集业务运行时的数据，做到有问题及时预警，方便分析定位。

异步开发框架 - 业务流程分析

普通业务 - 线性执行:



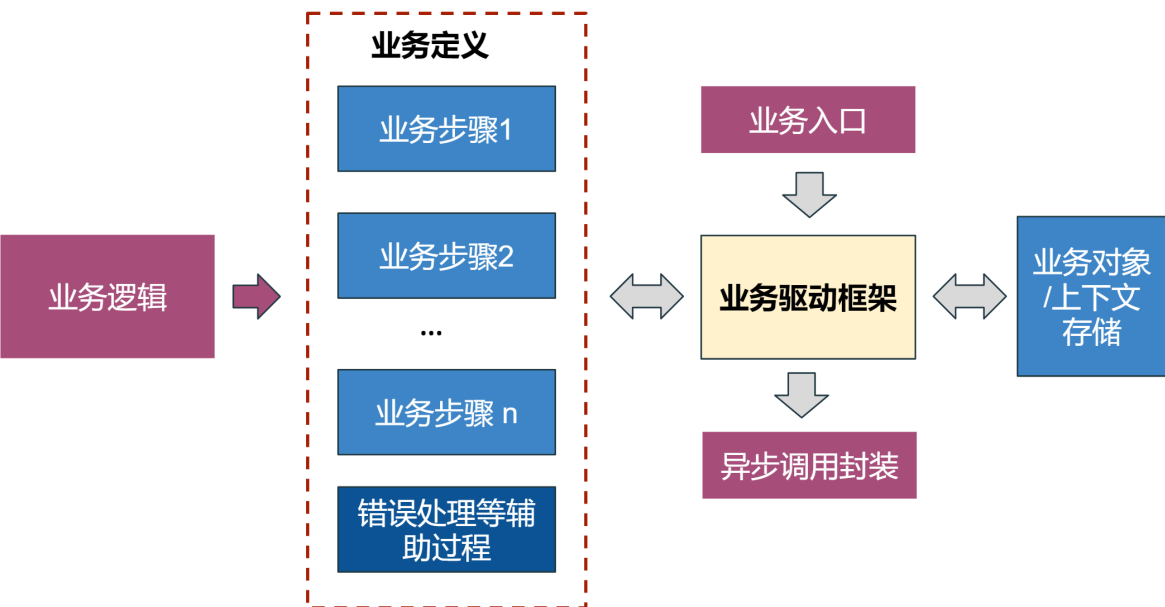
包含更复杂流程的业务:



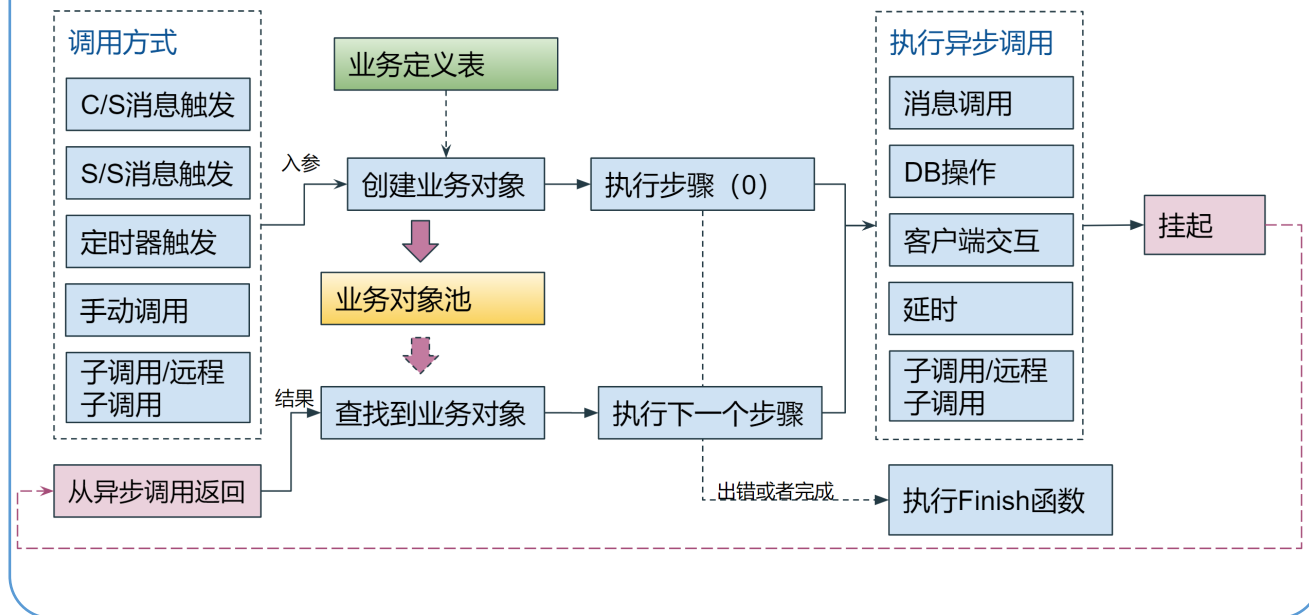
异步开发框架 – 方案原理

通过业务的执行特性分析，我们可以通过框架把这些过程有效组织起来

- 拆分业务的步骤 – 每个步骤可以看成一个原子操作 – 一口气执行完，中间不会停顿（同步过程）
- 将这些原子操作串起来。并通过框架提供异步调用的封装 – 类RPC
- 提供简单的调用，丰富的触发方式，可定制的控制原语。
- 框架提供了丰富的日志信息和监控数据，可帮助业务开发同学进行问题定位。



异步框架原理简图



异步开发框架 - 开发规范

开发规范和检查工具，确保业务逻辑正确书写

- 源代码的组织形式、文件格式的规范
- 函数编码的标准结构、使用预定义的检查宏
- 源代码扫描工具来排除潜在的问题

Step函数：

检查函数入口参数

可选：获取上下文数据指针

可选：判断是否重试调用入

检查CallData的类型是否正确

检查和处理调用返回的错误

业务逻辑代码段

执行异步调用或者结束业务

使用预定义的标准宏，方便使用和工具检查

```
static int async_start(LPASYNCBUSINESSBASE pstAsyncBase, LPASYNCCALLDATA pstCallData, char chIsRetry)
{
    ...CHECK_PARAM_RET(pstAsyncBase, ASYNC_RET_TERMINATE);
    ...LPASYNCBUSINESS_CLEARLOGIN pstClearLogin = ASYNC2USERDATA(ASYNCBUSINESS_CLEARLOGIN, pstAsyncBase);
    ...CHECK_PARAM_RET(pstClearLogin, ASYNC_RET_TERMINATE);

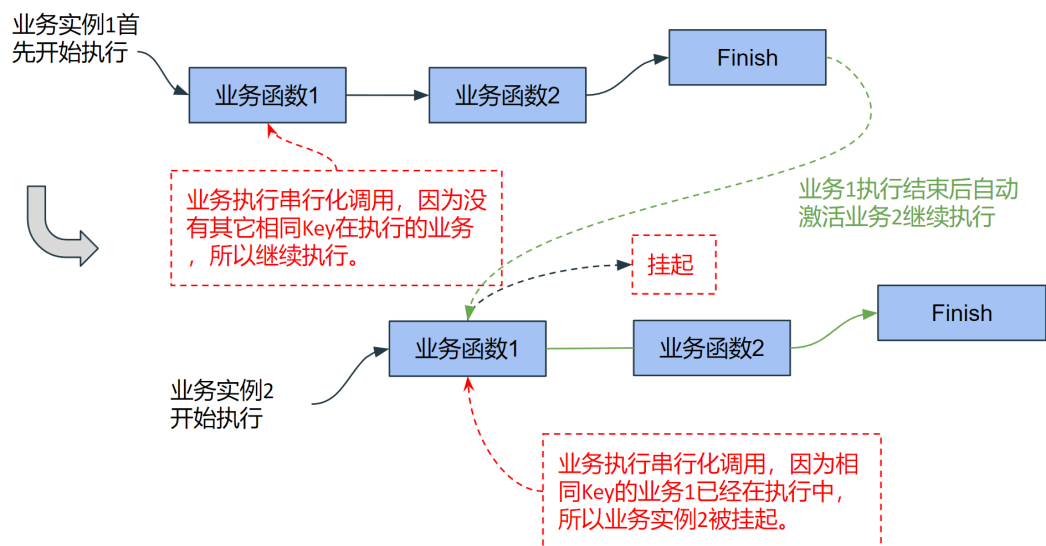
    if(0 == chIsRetry)
    {
        ...CHECK_ASYNC_CALLDATA_ANY_LEN_RET(pstCallData, sizeof(void*[2]), -1);
        ...// user data
        ...pstClearLogin->ullUin = *pullUin;
        ...pstClearLogin->iRecordVersion = *pRecordVersion;
    }

    ...// async query db
    ...SSMSG stSSMsg;
    ...init_async_business_db_insert(pstAsyncBase, &stSSMsg, DB_TABLE_ID_ROLE_IN_DB, DB_INSERT_TYPE_REPLACE, pstCle
    ...stSSMsg.stBody.stSSMsgC.stCData.stDBInsert.stCallBack.iRecordVersion = pstClearLogin->iRecordVersion;
    ...ASYNC_CALL_SS(pstAsyncBase, &stSSMsg, 10000);
}
```

异步开发框架 - 其它支持的特性

通过异步框架可以很容易做到，并不需要业务做很大的修改：

- ❑ 并发异步调用 – 支持一次异步内访问多个服务并收集返回数据。
- ❑ 业务并发控制 – 全局并发/基于Uin的并发控制
- ❑ 在线保持 – 为了确保某些重要的业务必需执行完成，可保持玩家在线状态，直到此类业务结束。
- ❑ 串行化支持 - 全局串行化/基于Uin的串行化控制
- ❑ 隐式调用 – 支持在执行过程中，如果其它的函数中调用了异步。可等待它执行完成再继续。可保证某些业务的顺序性。在没有异步框架前是无法做到的。

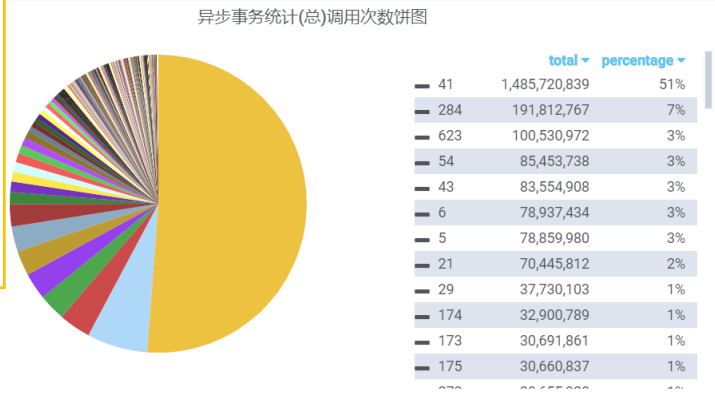


```
// 业务特殊标识
enum ASYNC_FLAG
{
    ... ASYNC_FLAG_NONE = 0,
    ... ASYNC_FLAG_PLAYER_UNIQUE = 1, ... // 同一时间每个角色唯一业务
    ... ASYNC_FLAG_GLOBAL_UNIQUE = 2, ... // 同一时间全局唯一业务
    ... ASYNC_FLAG_KEEP_LOGIN = 4, ... // 业务未结束前，保持角色在线
    ... ASYNC_FLAG_LOGOUT_REMOVE = 8, ... // 角色下线时自动结束此业务
    ... ASYNC_FLAG_LOGOUT_NOTIFY = 16, ... // 角色下线时通知此业务
    ... ASYNC_FLAG_KEEP_RUNNING = 32, ... // 业务未结束前不能停机
    ... ASYNC_FLAG_SINGLE_STEP = 0x40000000, ... // 单步业务，可通过此标识优化
    ... ASYNC_FLAG_AUTO_RETURN = 0x80000000, ... // 完成时自动通知父业务
};
```

异步开发框架 - 日志和监控



- 业务存量监控
- 内存使用监控
- 业务耗时监控
- 错误码监控
- 超时情况
- 每个Step的执行情况
-

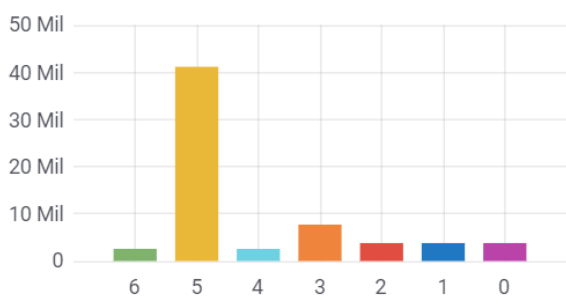


UID

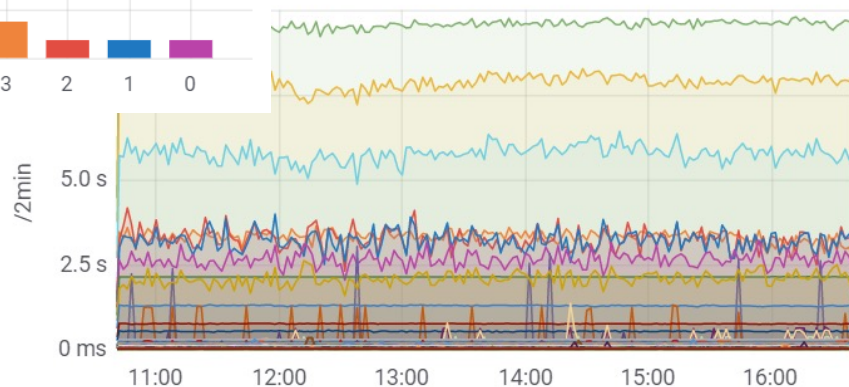
异步ID

```
[20210129 16:19:44.339315][520]<DEBUG> [72058810816699480] (async_upload_rank_data_start:1071) [1191-2-1-243.0] upload rankid:4 score:106
[20210129 16:19:44.339320][521]<DEBUG> [72058810816699480] (get_rank_report_sequence:1240) [1191-2-1-243.0] update rank:4 seq:119
[20210129 16:19:44.390057][522]<DEBUG> [72058810816699480] (async_upload_rank_data_result:1150) [1191-2-1-243.1] upload rank:4 success
[20210129 16:19:44.440694][523]<DEBUG> [72058810816699480] (async_end_occupy:1281) async release type [1191] id [5363822341644091635]
[20210129 16:19:44.440714][524]<DEBUG> [72058810816699480] (async_business_tick_one:3865) async wake type [1191] id [5363822341644091636]
[20210129 16:19:44.440725][525]<DEBUG> [72058810816699480] (async_occupy:1232) [1191-2-1-244.0] async occupy type [1191] id [5363822341644091637]
[20210129 16:19:44.440733][526]<DEBUG> [72058810816699480] (async_upload_rank_data_start:1071) [1191-2-1-244.0] upload rankid:4 score:107
[20210129 16:19:44.440743][527]<DEBUG> [72058810816699480] (get_rank_report_sequence:1240) [1191-2-1-244.0] update rank:4 seq:120
[20210129 16:19:44.481458][528]<DEBUG> [72058810816699480] (async_upload_rank_data_result:1150) [1191-2-1-244.1] upload rank:4 success
[20210129 16:19:44.491578][529]<DEBUG> [72058810816699480] (async_business_tick_one:3865) async wake type [1191] id [5363822341644091637]
[20210129 16:19:44.491588][530]<DEBUG> [72058810816699480] (async_end_occupy:1281) async release type [1191] id [5363822341644091638]
[20210129 16:19:44.491594][531]<DEBUG> [72058810816699480] (async_occupy:1232) [1191-2-1-245.0] async occupy type [1191] id [5363822341644091639]
[20210129 16:19:44.491598][532]<DEBUG> [72058810816699480] (async_upload_rank_data_start:1071) [1191-2-1-245.0] upload rankid:4 score:107
[20210129 16:19:44.491605][533]<DEBUG> [72058810816699480] (get_rank_report_sequence:1240) [1191-2-1-245.0] update rank:4 seq:121
[20210129 16:19:44.522110][534]<DEBUG> [72058810816699480] (async_upload_rank_data_result:1150) [1191-2-1-245.1] upload rank:4 success
[20210129 16:19:44.542335][535]<DEBUG> [72058810816699480] (async_end_occupy:1281) async release type [1191] id [5363822341644091637]
[20210129 16:19:44.542346][536]<DEBUG> [72058810816699480] (async_business_tick_one:3865) async wake type [1191] id [5363822341644091638]
[20210129 16:19:44.542353][537]<DEBUG> [72058810816699480] (async_occupy:1232) [1191-2-1-246.0] async occupy type [1191] id [5363822341644091639]
[20210129 16:19:44.542358][538]<DEBUG> [72058810816699480] (async_upload_rank_data_start:1071) [1191-2-1-246.0] upload rankid:4 score:108
[20210129 16:19:44.542375][539]<DEBUG> [72058810816699480] (get_rank_report_sequence:1240) [1191-2-1-246.0] update rank:4 seq:122
[20210129 16:19:44.582992][540]<DEBUG> [72058810816699480] (async_upload_rank_data_result:1150) [1191-2-1-246.1] upload rank:4 success
[20210129 16:19:44.593119][541]<DEBUG> [72058810816699480] (async_end_occupy:1281) async release type [1191] id [5363822341644091638]
```

Step执行次数

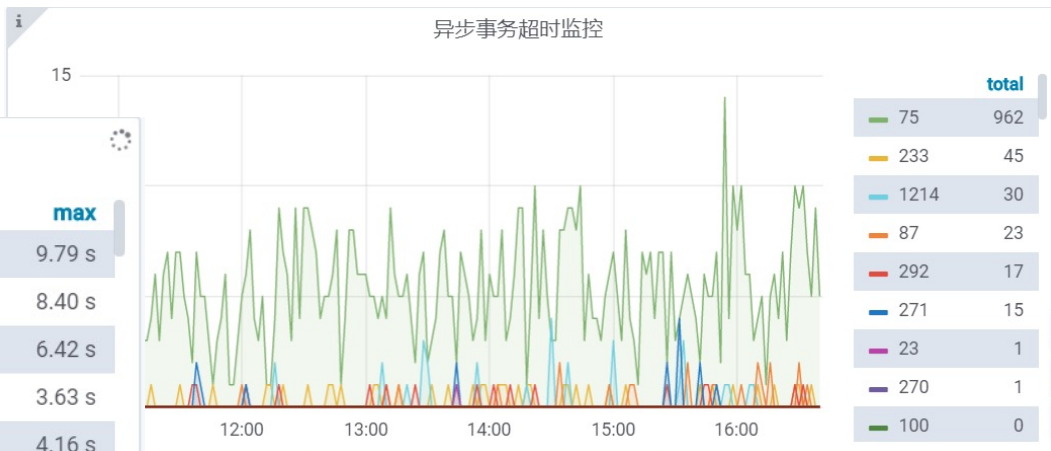


异步事务平均耗时(max)



max	
87	9.79 s
72	8.40 s
71	6.42 s
1157	3.63 s
191	4.16 s
192	3.98 s
83	3.20 s

异步事务超时监控



total	
75	962
233	45
1214	30
87	23
292	17
271	15
23	1
270	1
100	0

异步开发框架 - 总结



扩展数据框架 – 角色非核心数据存储方案

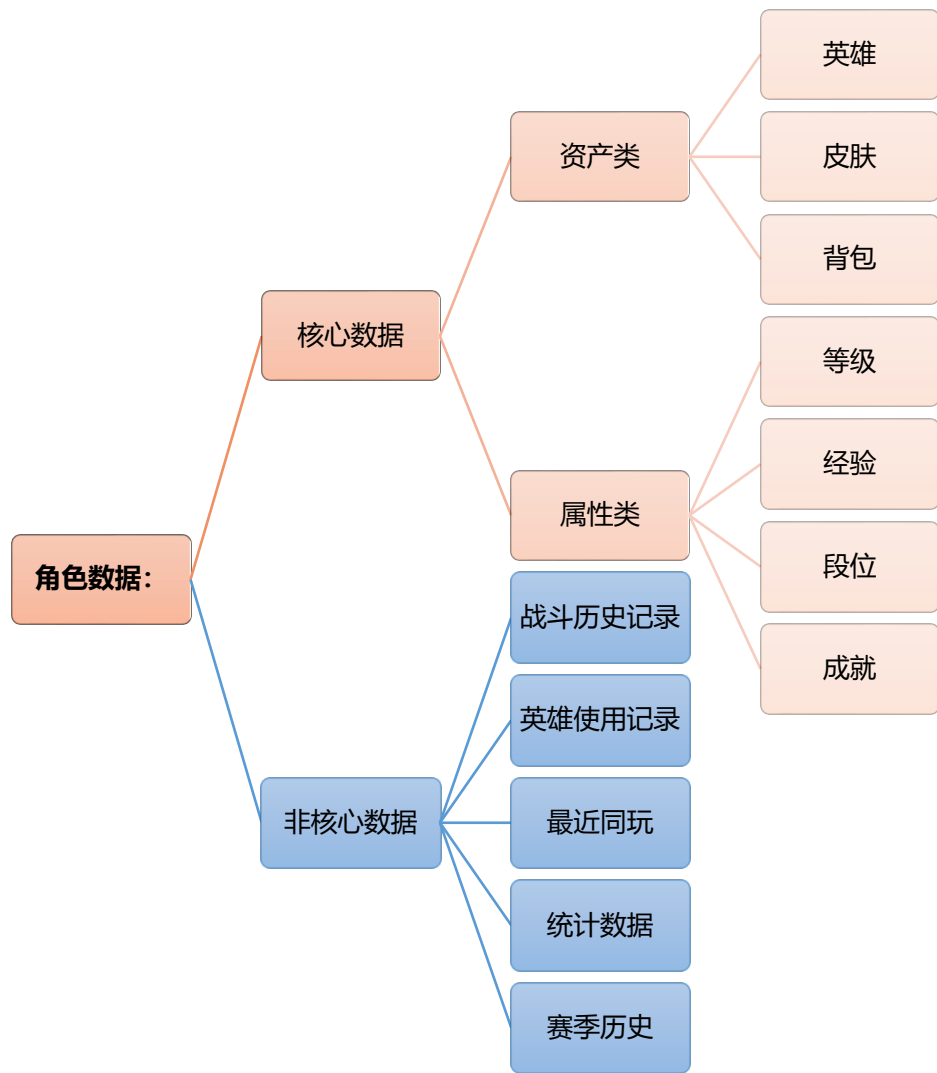


角色数据存储的问题：

- 角色数据膨胀 - 超过Tcaplus的限制
- 登录时加载角色数据，性能差
- 过多的数据将影响内存的利用

解决方案：

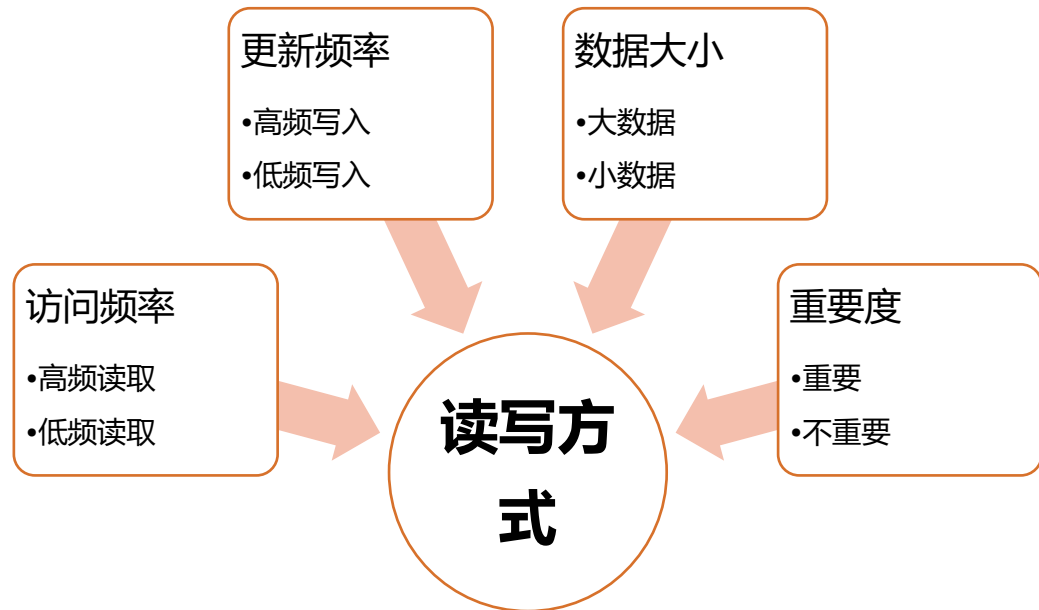
- 拆分玩家的核心数据和非核心数据
- 核心数据保存在 RoleInDB，保持登录即加载的逻辑
- 非核心数据分离存储，按实际需要读取和修改



RoleExtDB:



扩展数据框架 – 读写设计



通过综合的考量，来确定数据的读取方式以及一致性保障的方法

何时加载？

- 登录时
- 需要时

是否需要Cache？

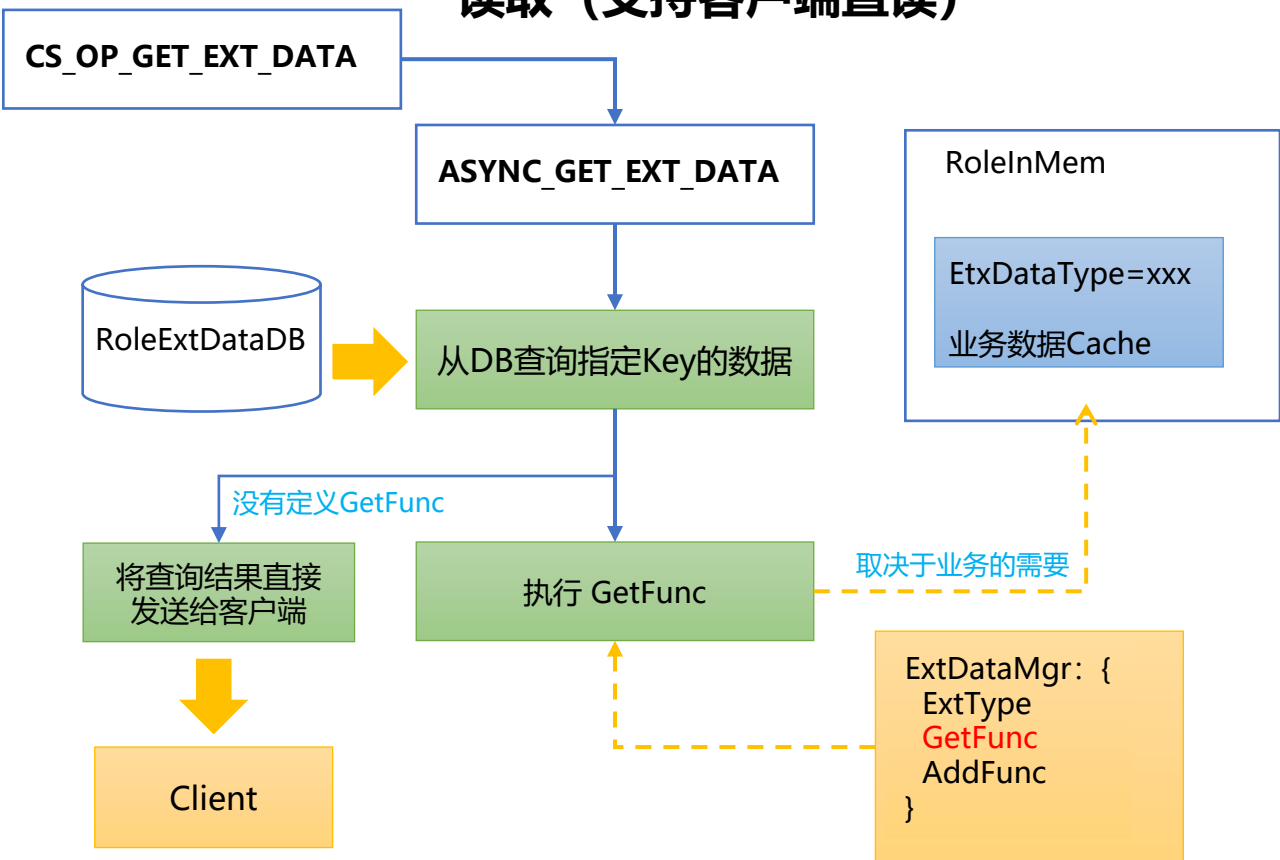
- 不Cache
- 全量Cache
- 部分Cache

是否需要一致性保障？

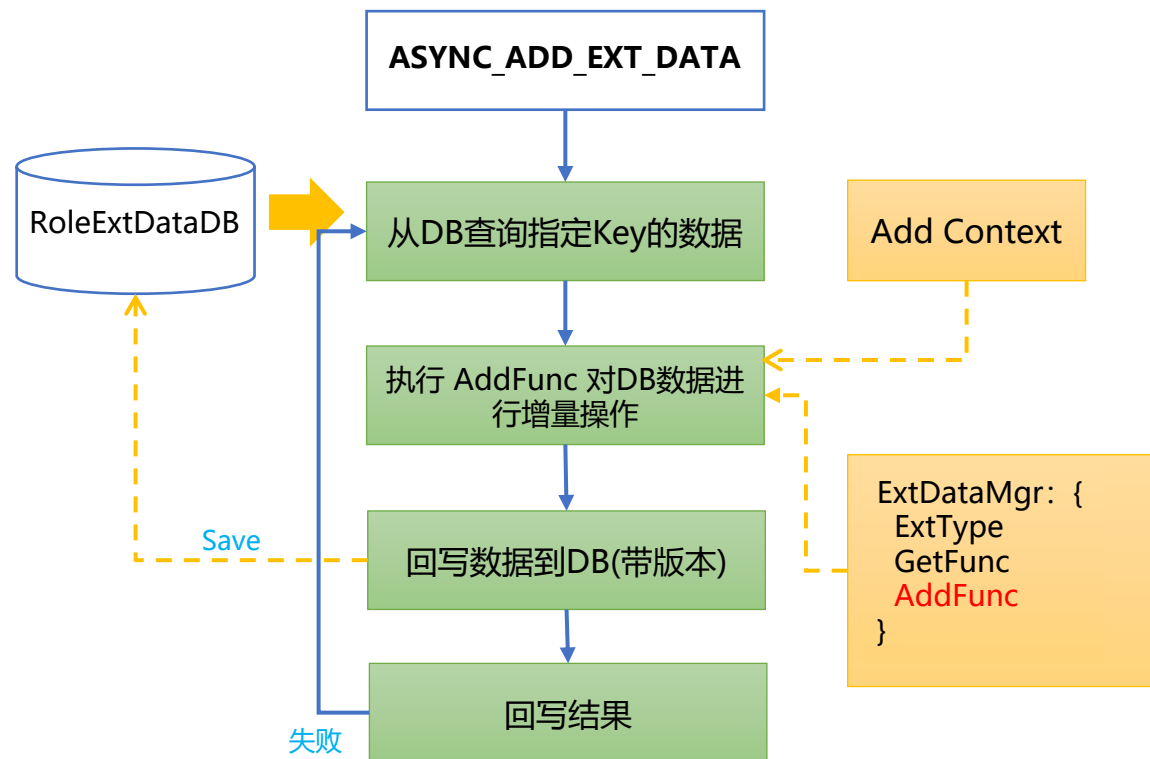
- 写入即可，可丢失
- 保证一致

扩展数据框架 - 读写方案图

读取 (支持客户端直读)



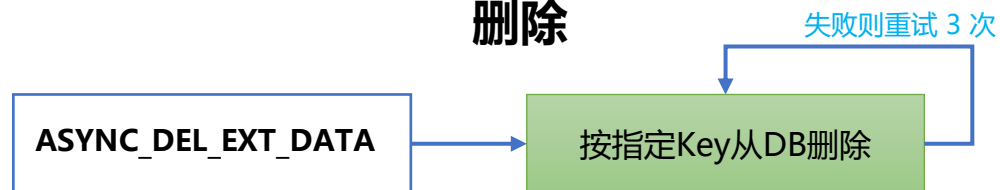
增量修改



覆盖更新



删除



动态数据配置 - 问题



线上环境对配置的灵活调整

- 商品价格的线上调整
- 商品上下架时间
- 运营活动配置
- 临时发布的公告



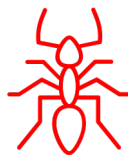
周期性或者按需功能的开放和配置

- 娱乐模式的及时开放
- 限免英雄的配置
- 活动皮肤池的配置



灰度验证新的功能或者参数调整

- 网络参数优化时线上的灰度验证
- 匹配算法调整时的灰度验证
- 改进协议打解包算法后线上的灰度验证



应对线上的紧急情况

- 功能出现了异常时紧急关闭相关入口
- 消息处理中存在Bug, 紧急屏蔽相应的ID
- 界面UI某个元素显示存在问题, 紧急隐藏

共同特点

数据需要便于修改

修改后能快速生效

可以即时下发给客户端

方案



通过大管家或者业受来配置,
由服务器下发给客户端

满足需求

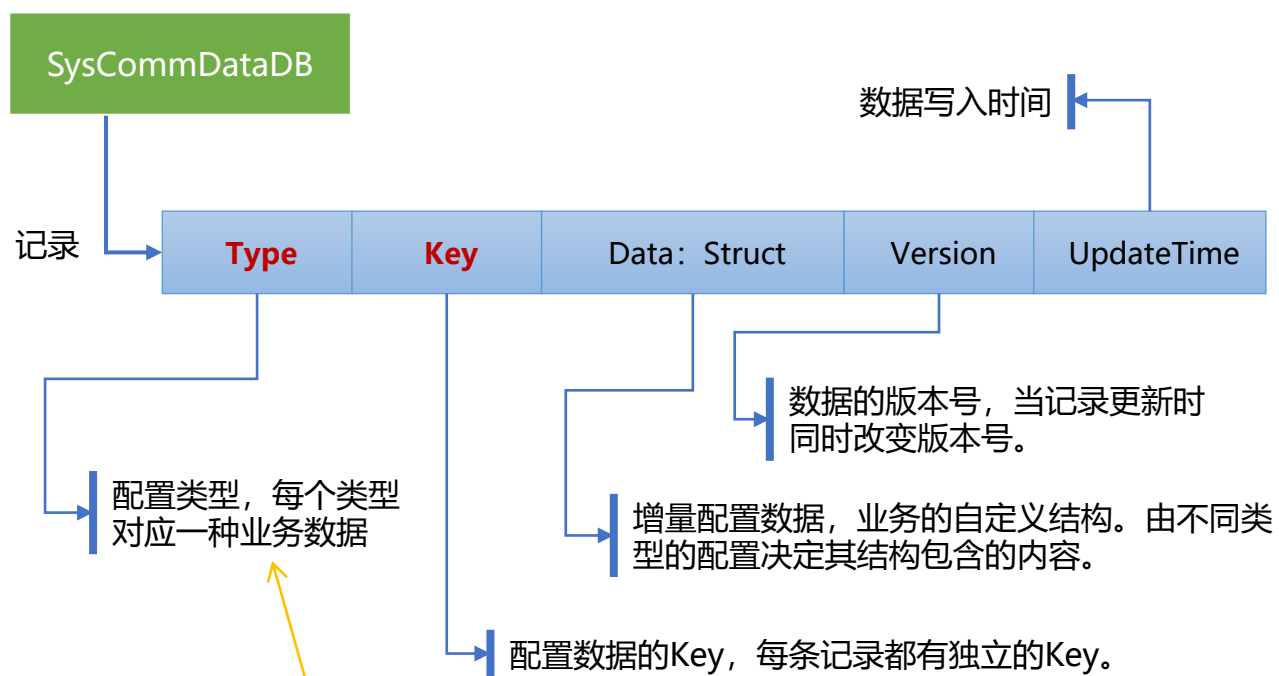


通过热更来处理

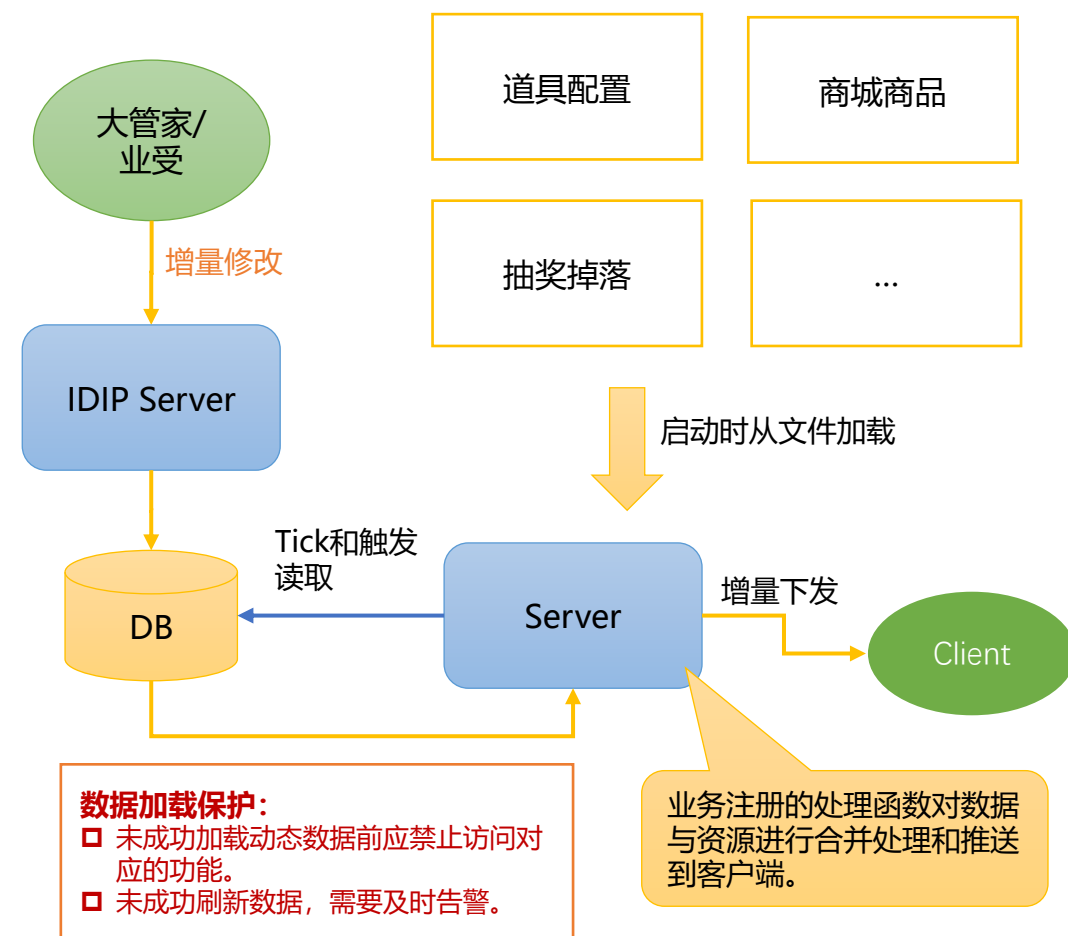
工作量大,
时间长,
客户端需要退出
下载才能生效

动态数据配置 - 框架

需要动态配置的数据类型比较多，我们从存储结构和操作逻辑上通过一个统一的框架来处理，而将业务具体的设计抽取出来。数据按Type+Key的方式统一存储在SysCommDataDB表内。



其中一个特殊的配置类型为：Comm_Update_Info。当任意动态数据配置修改，则同时更新Update_Info对应的Key（为修改的Type）的Version和UpdateTime。它用于配置的定时刷新逻辑。



动态数据配置 - 开关系统

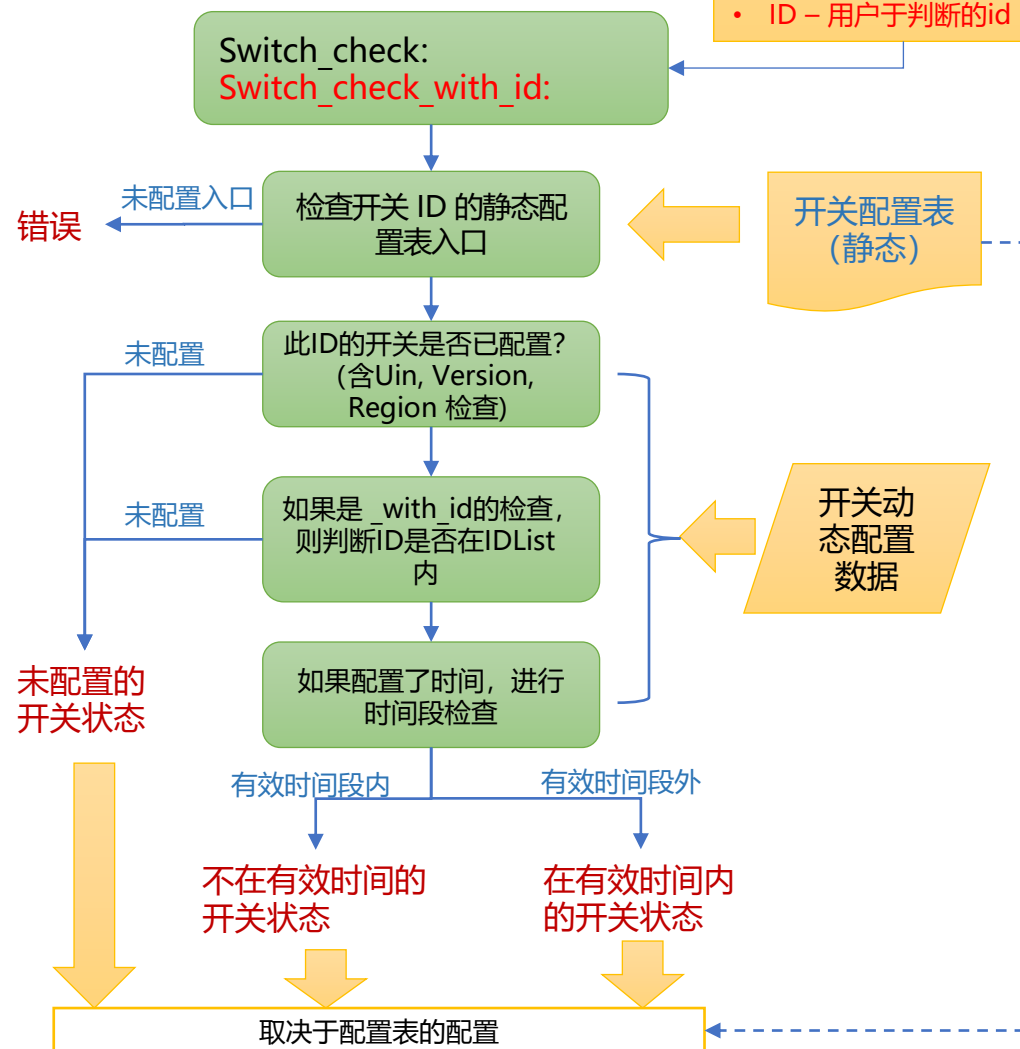
用于开关判断的参数:

- Id - Key 开关的ID
- Region - 地区/分区
- Version - 客户端的版本
- Uin - 玩粗ID
- ID - 用户于判断的id

开关系统的动态配置数据:

字段	说明
ID	开关的唯一ID, 每个ID的开关对应一项功能的配置。
Params	开关参数, 最大3个, Int。由开关自定义使用。
PlayertList	UIN的列表, 最大100个, 如果此开关只针对某部分玩家开启, 则配置此列表。它也可以用来做小范围的灰度或者针对特定玩家的处理。
Context	一个字符串, 由开关自定义使用方式。
StartTime, EndTime, LoopType	开关的生效时间。由LoopType来决定时间的配置方式。是按绝对时间段, 还是每日的相对时间段。更复杂的时间控制可以由特定开关的业务自己来实现。
IDList	ID列表, 默认的情况下可以判断开关是否启用时带上ID。当ID位于IDList中时, 认为开关针对此ID是启用的。比如用于某个英雄是否为当前配置的限免英雄列表内。
GrayList	灰度参数, 最大5个。默认灰度规则为如果只配置1个灰度参数, 则按Uin尾数 (模10000) 0~参数GrayList[0]的角色。如果配置为2个参数则按Uin尾数 (模10000) GrayList[0]~GrayList[1]之间的角色。
Version	指定此开关只匹配此版本及之后的客户端。低于此版本的客户端相当于未配置。
ResVersion	指定此开关只匹配此资源版本及之后的客户端。低于此版本的客户端相当于未配置。

- 结合静态的开关配置表, 我们可以确定每个开关在未配置时, 已配置在生效时间范围内, 以及在生效时间范围外分别对应的开关状态为 **ENABLE** 还是 **DISABLE**。
- 是否下发到客户端也是通过开关配置表来配置的。
- 所有的开关配置参数可以使用默认的判断, 也可以自定义判断逻辑。



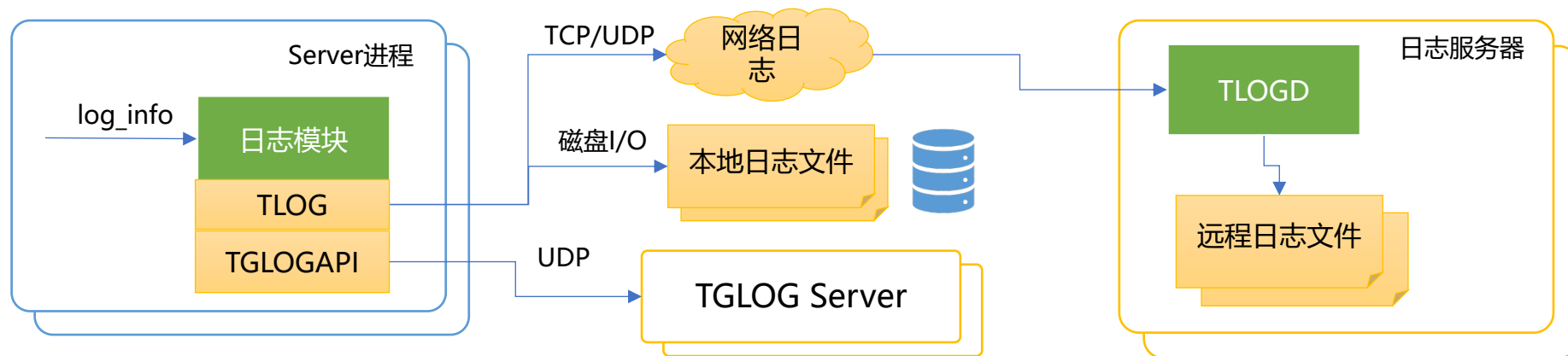
动态数据配置 – 设计上的注意点



动态数据配置对产品的正确运行有较大的影响，我们在设计上需要高度关注以下问题：

- ✓ 如果关系到核心的数据，要确保进程在正确加载后才能提供对外服务。如商品价格，赛季开启时间。
- ✓ 如果长时间不能成功刷新配置数据，需要立刻告警，及时处理。
- ✓ 要做好数据校验，如价格，折扣等要和客户端的上行对比。必要时刷新客户端的配置。
- ✓ 数据最好是通过配置时间来生效的，因为刷新可能存在延时或者不完全同步。
- ✓ 数据的合并方式，要能恢复原始数据。因为动态配置可能会被删除或者过期。
- ✓ 每次配置修改都要有Version的修改来标识。确保修改能被正确识别。这里不能直接使用Tcaplus的记录版本号，因为删除再插入会重置为1。

日志系统的改造



Lgame有3种日志:



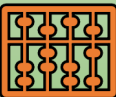
运行日志:

程序运行时输出的日志，写入本地文件，或者写入日志服务器，运行日志可分模块和级别写入不同文件，如Error级别日志。也支持Uin染色。



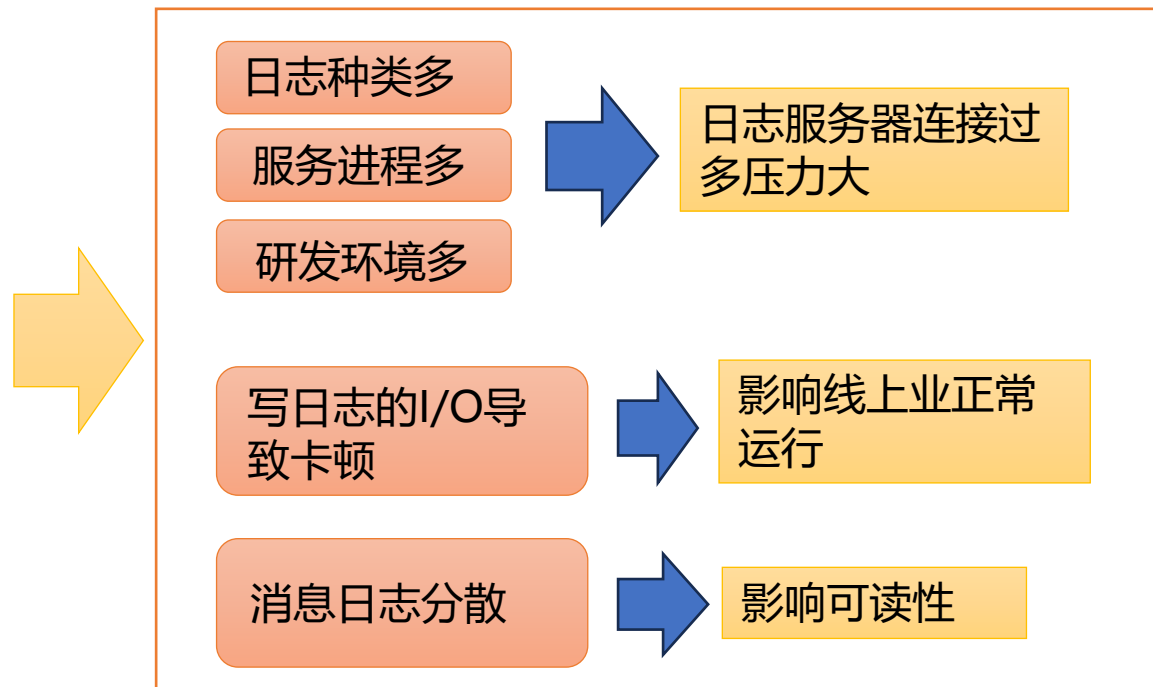
消息日志:

服务之间以及客户端和服务端之间的消息收发以及消息内容的详细日志，用于调试。线上如果某玩家异常，也可以打开详细日志用于分析。

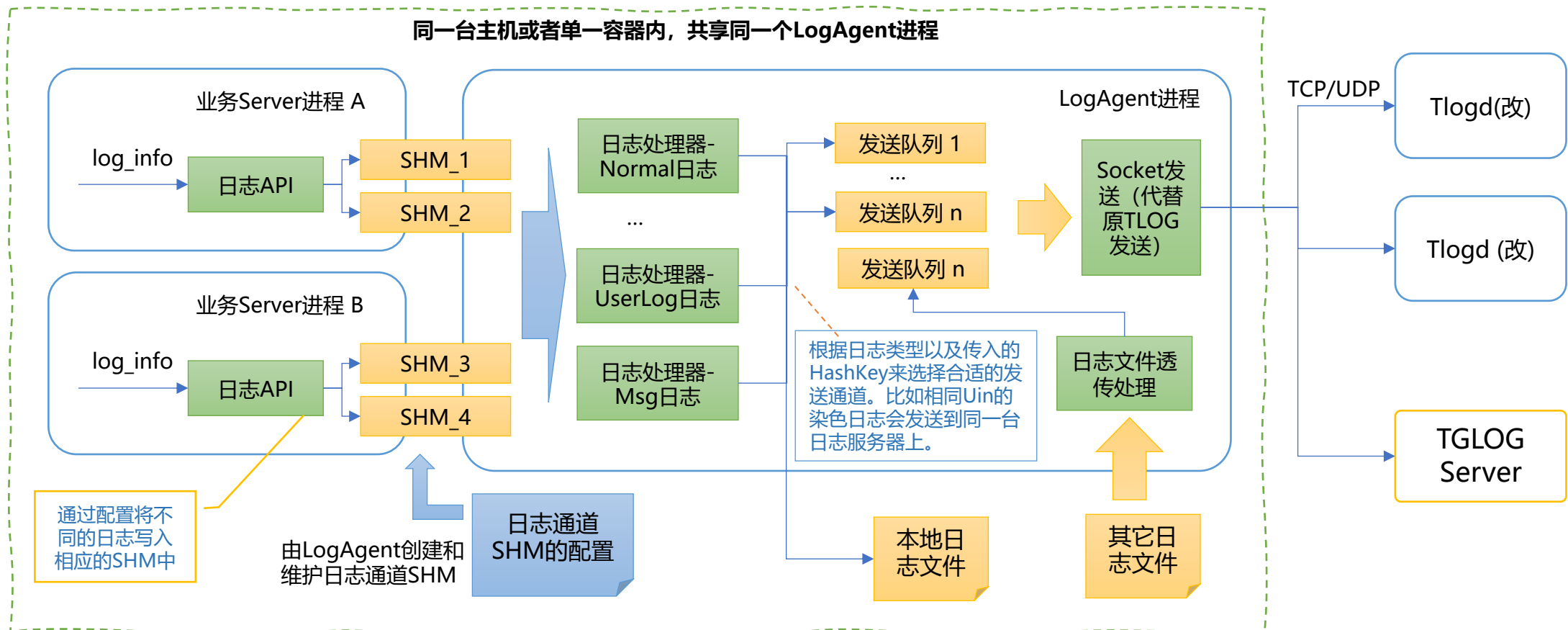


埋点日志:

经分的埋点数据，通过TGLOG发向经分日志服务器。为方便调试，本地也可以写副本。



日志系统的改造 - LogAgent

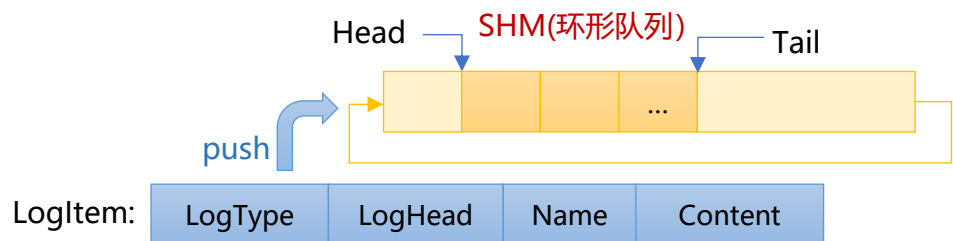


业务日志如何选择SHM通道：

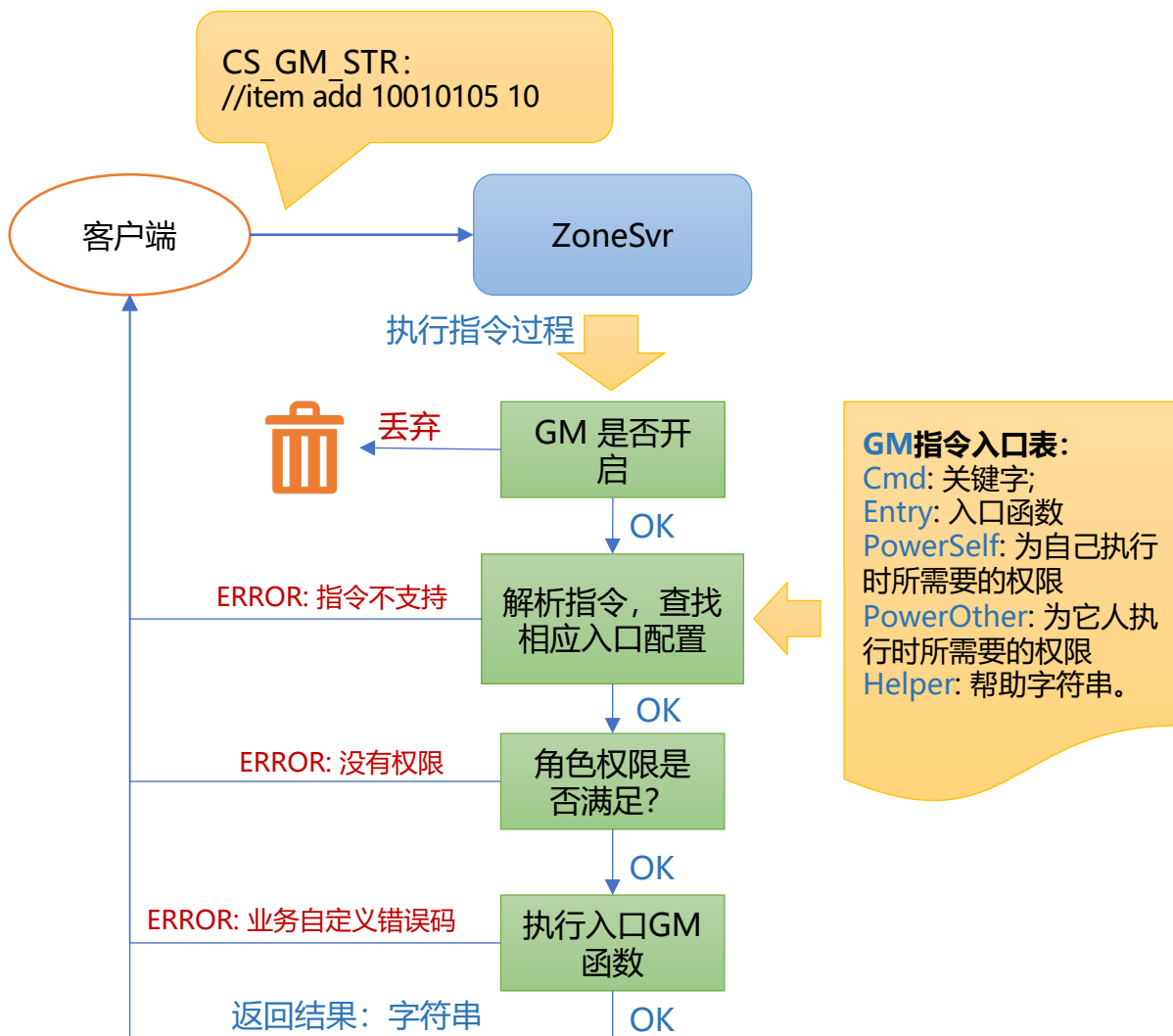
- ❑ 重要日志独立通道，防止丢失。如埋点。
- ❑ 根据日志量的大小来共享通道。
- ❑ 均衡使用各通道，保持较高的利用率。

优化措施：

- ✓ 消息日志业务服务器只传递原始数据，由LogAgent来进行格式化等操作，提升性能。
- ✓ 客户端的日志，如埋点消息，由LogAgent统一处理。ZoneSvr只需要透传，不用解包。



跨服务的GM框架设计 – 老框架的问题

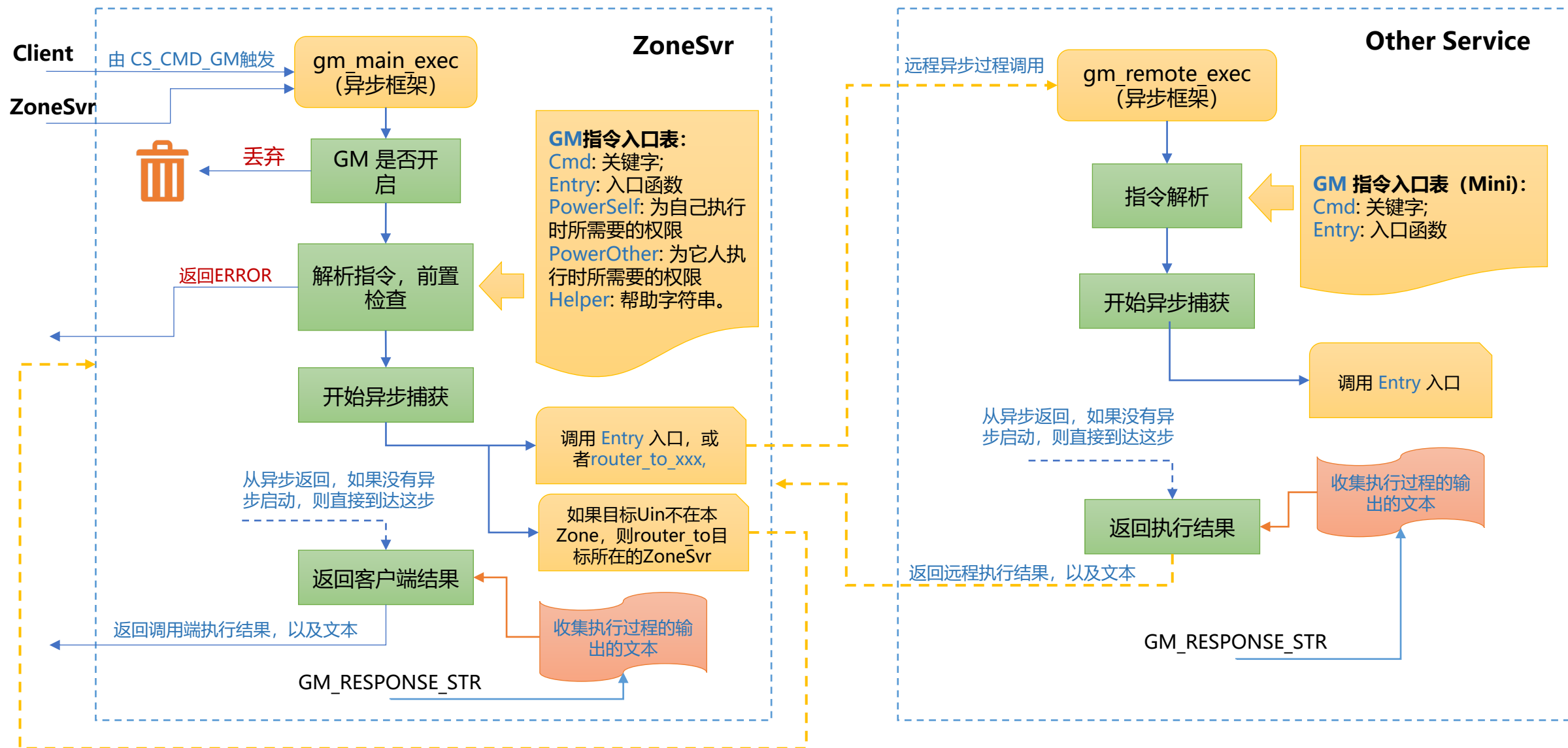


痛点问题:

- **GM 指令只能在角色所在的ZoneSvr上执行**
 - 在微服框架下, 业务分散到多个服务。很多都有支持GM的需求。
 - 手动编码在各服务间传递GM指令是很低效的。
- **GM指令不支持异步, 返回结果不准确**
 - 带有异步的调用时, GM的返回结果不准确。对于工具不友好。
 - 异步执行时。想要在后续的处理中继续返回执行结果也很难处理。

思考: GM是一个通用的逻辑, 我们希望能通过一个框架来解决以上问题。让GM的开发过程变得非常的简单和高效。

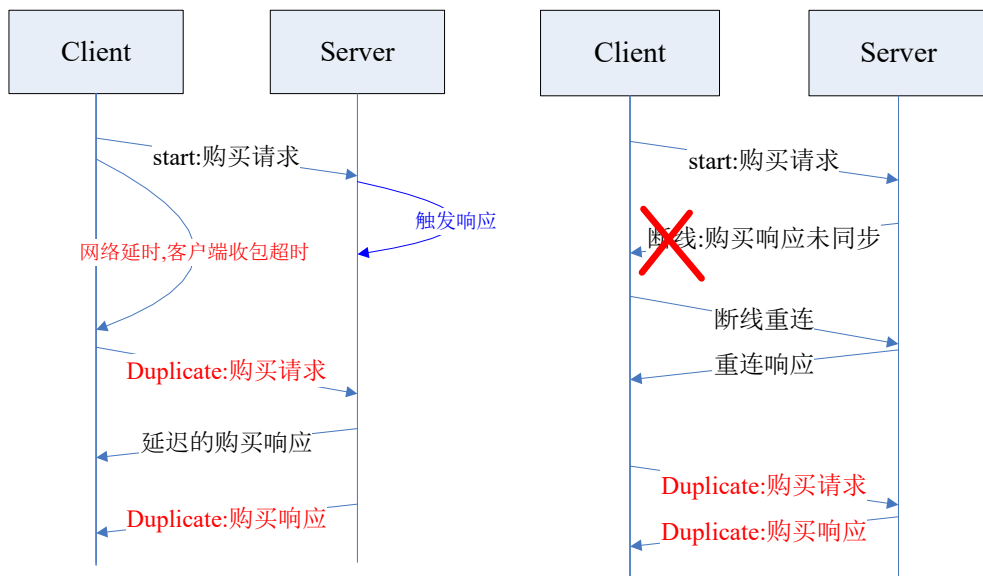
跨服务的GM框架设计 – 新的设计



弱网综合解决方案 - CmdSeq机制

C/S之间的网络问题：

- 请求丢失 - 请求没有完成
- 响应延迟 - 请求可能已经完成
- 重复点击 - 手抖，一次买了两个...
- 高频访问 - 恶意攻击，引起服务器性能问题
- 超时无响应 - 客户端长时间转圈或者卡住
- ...



CmdSeq机制：

请求Msg：

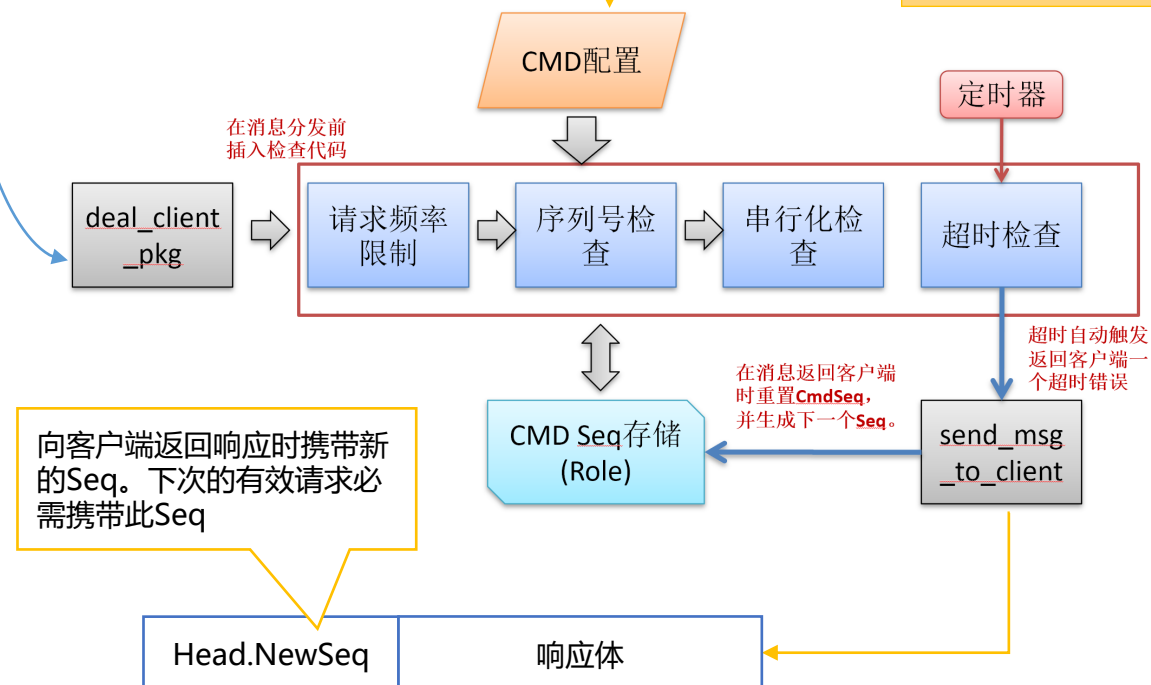


向服务器发送请求时需要携带此Seq。

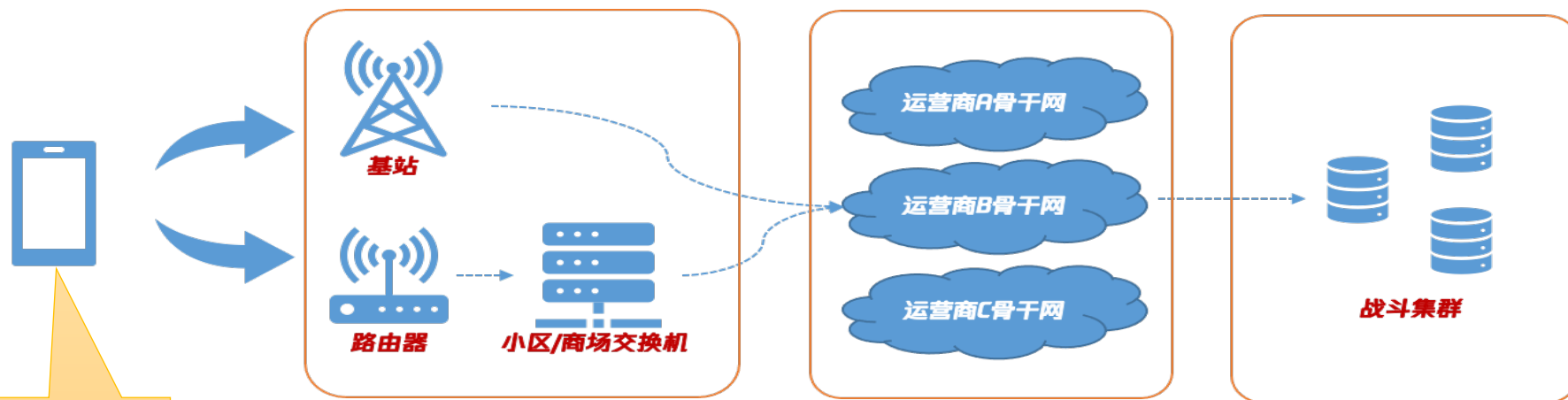
服务器：

Cmd x: 1
Cmd y: 99

每个需要检查的Cmd都有一个对应的请求Seq，会下发给客户端。



弱网综合解决方案 – 战场网络优化



客户端优化:

客户端通过线程分离, 预表现, C++改造等手段, 有效提升了玩家对网络的体验。

优化连接链路:

- ✓ UDP直连, 减少中间转发环节
- ✓ 多端口监听, 提升客户端连接成功率
- ✓ 优化流量挤占, 战场内延迟大厅消息的传输
- ✓ MTU探测, 避免UDP分包
- ✓ 多机房部署, 最优接入

优化帧同步算法:

- ✓ 上下行动态冗余, 基于玩家网络的冗余窗口调整
- ✓ 快速重传, 借鉴RTT算法来优化补帧时间, 降低补帧延时
- ✓ 双倍发包, 优化丢包严重的场景
- ✓ 协议压缩/裁剪, 通过训练提升算法压缩率

应对网络抖动:

- ✓ TCP/UDP双通道, 当发生抖动时往往TCP和UDP不会同时抖动
- ✓ 4G&WIFI双发, 充分利用玩家设备的多网络优势, 在抖动发生时通过4G通道来快速恢复帧数据。

服务器底层优化:

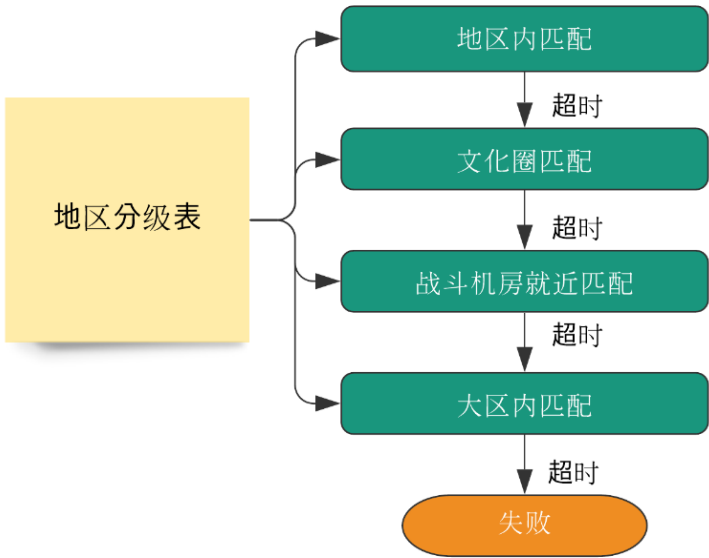
- ✓ 磁盘IO分离, 杜绝因IO引起的CPU卡顿。
- ✓ CPU绑核, 降低对CPU核心的调度。
- ✓ 负载预测, 基于战场状态来预测未来进程的负载, 防止产生性能瓶颈。



数据驱动优化, 提升网络优化效率。

另外: 通过Anycast加速, 运营商, 路由器厂商合作等, 给部分地区或者用户也带来明显的网络体验上的改善。

弱网综合解决方案 – 分区匹配与最优IDC选择



地区	地区名称	IDC1	IDC权重1	IDC2	IDC权重2	IDC3	IDC权重3
LK	斯里兰卡	斯德哥尔摩战斗服	1	巴林战斗服	1	法兰克福战斗服	10
MV	马尔代夫	斯德哥尔摩战斗服	1	巴林战斗服	10	法兰克福战斗服	1
NP	尼泊尔	斯德哥尔摩战斗服	1	巴林战斗服	10	法兰克福战斗服	1
LA	老挝	曼谷战斗服	10	新加坡战斗服	7	香港战斗服	2
TH	泰国	曼谷战斗服	10	新加坡战斗服	7	香港战斗服	5
VN	越南	曼谷战斗服	10	新加坡战斗服	5	香港战斗服	7
KH	柬埔寨	香港战斗服	10	新加坡战斗服	8	曼谷战斗服	6
PH	菲律宾	香港战斗服	10	新加坡战斗服	7	曼谷战斗服	3
HK	香港	香港战斗服	10	东京战斗服	7	曼谷战斗服	3
MO	澳门	香港战斗服	10	新加坡战斗服	7	曼谷战斗服	3
CNROW	中国大陆使用香港战斗服	香港战斗服	10	新加坡战斗服	7	曼谷战斗服	3
TW	中国台湾省	台湾战斗服	10	香港战斗服	7	东京战斗服	3
JP	日本	东京战斗服	10	汉城战斗服	5	香港战斗服	1
KR	韩国	汉城战斗服	10	东京战斗服	5	香港战斗服	1
MM	缅甸	新加坡战斗服	10	曼谷战斗服	7	香港战斗服	3
TL	东帝汶	新加坡战斗服	10	香港战斗服	2	曼谷战斗服	7
BN	文莱	新加坡战斗服	10	香港战斗服	5	曼谷战斗服	8
MY	马来西亚	新加坡战斗服	10	曼谷战斗服	7	香港战斗服	6
SG	新加坡	新加坡战斗服	10	曼谷战斗服	7	香港战斗服	5
ID	印尼	印尼战斗服	10	新加坡战斗服	8	曼谷战斗服	7
ES	西班牙	法兰克福战斗服	10	斯德哥尔摩战斗服	5	巴林战斗服	1

战斗机房就近部署

我们根据本球网络的分布情况，结合腾讯云以及AWS等的基础建设，选择全球网络质量较佳的一些点部署战斗机房，尽可能保证全球的玩家的最佳体验。

匹配按地理位置分级搓合

当玩家开始匹配时，我们按玩家登录所在的区域分级匹配，首先在相同的区域内，如果超过配置的时间不能成功匹配，则升级为上一级区域，直到整个大区。

根据局内玩家的加权综合优选战斗机房

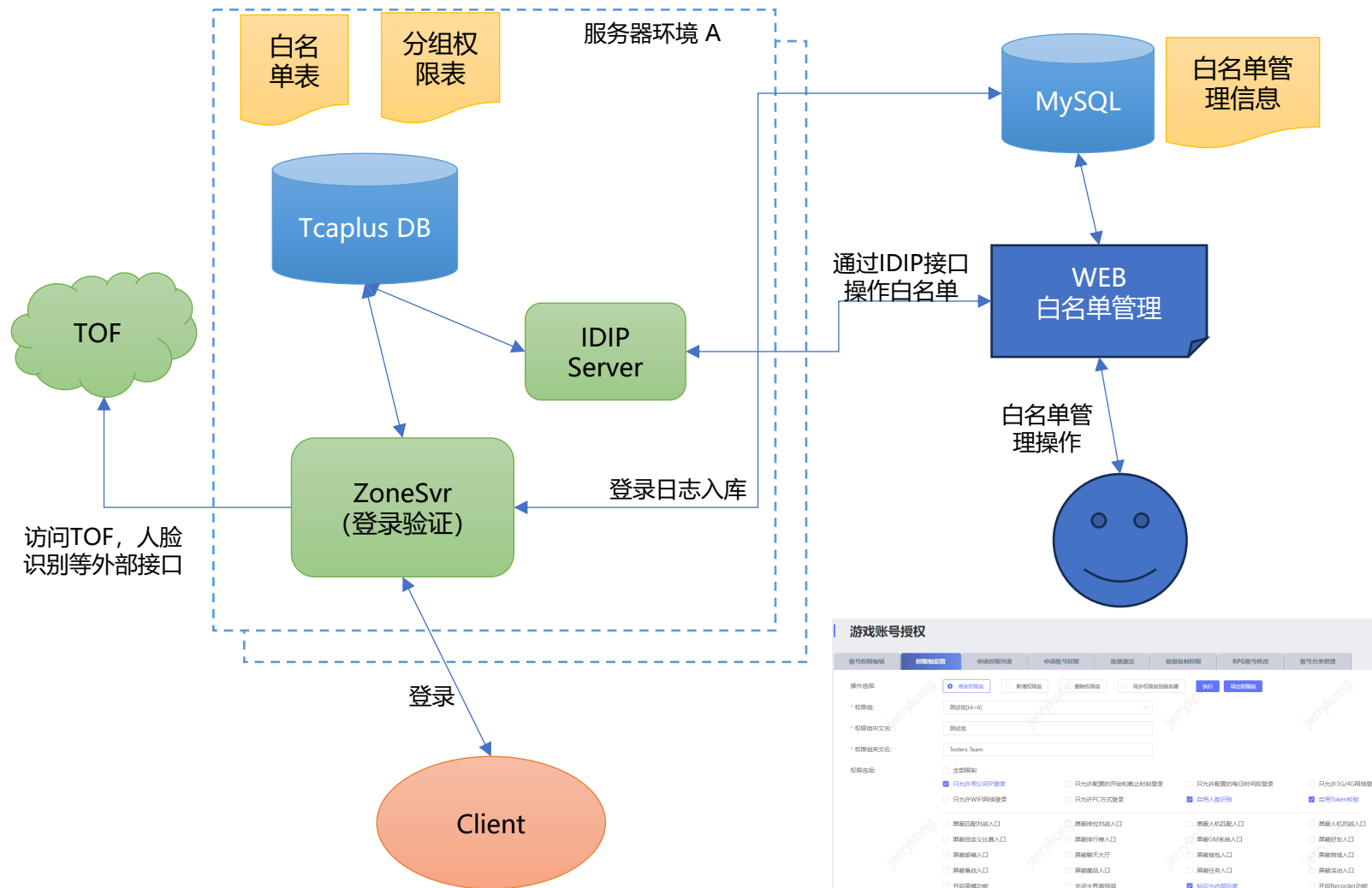
一局战斗的玩家最终会分配到哪个机房来创建战场，是通过当局所有玩家进行加权计算，并从中选择一个权重最大的IDC。这个就是综合对所有人最优的IDC。

通过地理位置的分级搓合，以及战斗机房的就近分配来尽量满足不同地区玩家，尽乎一致的游戏体验。

服务器登录安全 – 研发期保密以及封闭测试

服务器登录安全的手段

- 白名单控制
 - 分级权限：不同的帐号可以访问的内容不一样，不同帐号的密保方式不一样，...
 - 分环境控制：不同环境的白名单可以是不一样的。
 - 分组管理：通过分组来简化权限管理，需要对权限表的热Key进行Cache
- 登录PIN码认证
 - 自定义Openid+密码：用于个人开发机等
 - RTX名+PIN码企业认证：用于公共体验和测试等环境。
- 设备绑定
 - 只允许指定的设备登录：内部设备先登记绑定才能登录
 - 只允许绑定的设备登录（第一次登录时绑定到设备）：对外封测时，不能预先绑定设备。
- IP地址绑定：防止在开发区域外部登录游戏，导致泄露。
- 人脸认证：防止帐号出借或者泄露。
- 登录记录回扫：事后的审核。



thanks!

LEARNING EMPOWERS YOUR FUTURE



Tencent
腾讯学堂